Advanced Web Development

3 - Next.js & Redux Toolkit (RTK)

Jeannette Chin

Learning Objectives

- Next.js Basics
- Rendering Strategy
- Routing and Navigation
- Client and Server Components
- Data Fetching – client and Server
- useEffect hook
- Redux Toolkit for global state management

Teaching code – Part 1

https://stugit.cmp.uea.ac.uk/jc/Adv-WebDev-W3-part1.git

Next.js

- REACT framework created by Vercel
- Building full-stack web applications
- Hybrid Rendering support Server-side rendering (SSR), Static Site Generation (SSG), and Incremental Static Regeneration (ISR) out of the box
- Underneath the hook, it’s built on top of node:
  - Development and build tooling, uses node to run its CLI, compile code, bundle assets and execute static site generation (SSG)
  - In deployment, next.js runs as standard node HTTP server – server components, actions, API route handlers have all access to node built-in modules.

SSR (Server-Side Rendering)

- HTML for a page is generated on the server on every single user request.
- When a user visits a page, the server fetches the latest data, compiles the HTML on the fly, and sends the ready-to-render page back to the browser.
- **When HTML is generated:** At request time (every single page load).
- **Best for:** Pages with highly dynamic, personalised, or real-time data that changes continuously.

Example usage - Personalised User Dashboard / Feed

- **Scenario:** A user logs into a banking app or social media platform to view their personal newsfeed.
- **Why SSR?** The content is unique to *that specific user* and changes frequently. You cannot pre-build this page beforehand because you don't know who is visiting until they send their request (with session cookies or auth tokens).

SSG (Static Site Generation)

- HTML for the pages is generated **once at build time** (when you run `npm run build` or deploy your code).
- The static HTML and JSON files are then deployed to a Content Delivery Network (CDN) and served instantly to every visitor.
- **When HTML is generated:** At build time (before any user requests the page).
- **Best for:** Content that does not change often and is identical for every single user. It provides maximum speed, SEO efficiency, and low server cost.

Example Usage: Corporate Blog or Documentation Site

- **Scenario:** A developer documentation hub or a company blog with articles.
- **Why SSG?** An article written on Monday looks identical whether 10 people read it or 100,000 people read it on Friday. Generating the HTML once during deployment means visitors load pre-rendered HTML from a nearby CDN in milliseconds with zero server computation overhead.

ISR (Incremental Static Regeneration)

- is a hybrid strategy that combines the performance of static generation (SSG) with the flexibility of dynamic rendering (SSR).
- It allows you to **update static pages in the background after deployment** on a per-page revalidation timer without re-building the entire website.
- **When HTML is generated:** Built statically at build time, then automatically re-generated in the background when a request comes in *after* a designated time interval (e.g., revalidate every 60 seconds).
- **Best for:** E-commerce sites, large content platforms, or catalogue pages with thousands of items where data changes periodically, but real-time updates on every millisecond are not required.

Example Usage: E-Commerce Product Catalogue

- **Scenario:** An online store with 50,000 products where prices or inventory counts update every few hours or days.
- **Why ISR?**
- Running SSG for 50,000 pages would take hours to build on every minor price tweak.
- Running SSR on every page load would put massive compute strain on your server during peak sales traffic.
- With **ISR**, pages load at blazing-fast CDN static speeds. If a user visits a product page and the 60-second revalidation timer has expired, React serves the cached page instantly, fetches fresh inventory data in the background, updates the cache, and serves the new price to the *next* visitor.

Comparison Summary

Feature	SSR (Server-Side)	SSG (Static Site)	ISR (Incremental Static)
Generation Time	Every Request	Build Time	Build Time + Background Updates
Speed / CDN	Slower (Server compute required)	Fastest (Served directly from CDN)	Fastest (Static CDN + background update)
Data Freshness	100% Real-time	Fixed at build time	Periodically fresh (e.g., every X seconds)
Server Load	High	Lowest	Low
Primary Use Case	User Dashboards, Live Feeds	Blogs, Docs, Landing Pages	Product Catalogues, Large Sites

Why Next.js?

- Solves traditional Client-Side Rendering (CSR) React drawbacks efficiently.

```
<!DOCTYPE html>
<html>
  <head><title>My App</title></head>
  <body>
    <div id="root"></div> <!-- EMPTY! Content hasn't loaded yet -->
    <script src="/bundle.js"></script>
  </body>
</html>
```

- Search Engine Optimisation: Search engines crawl fully-rendered HTML content effortlessly.

Why Next.js?

- Performance Gains: Faster First Contentful Paint (FCP) and lower latency.
  - FCP - measures how long it takes from the moment a user clicks a link (or presses Enter) until the browser renders the very first piece of DOM content on the screen.
  - Google performance benchmarks: Good FCP – <=1.8s, needs improvement – 1.8s – 3s, poor - >30s
- Developer Experience: very little effort needed for setup, automated routing.

Next.js vs other Web Frameworks

Feature / Aspect	Next.js (React)	Node.js (Express)	Python (Django / FastAPI)
Primary Focus	Full-Stack UI & React SSR	Backend API & Microservices	Backend Services & Data APIs
Rendering	Server & Client Components	Client-Side / Engine Templates	Server Templates / API JSON
Routing Mechanism	File-system based (App Router)	Code-based explicit routes	URL dispatcher config
Full-Stack Integration	Seamless single codebase	Requires separate React app	Requires separate React frontend

Next.js Concept

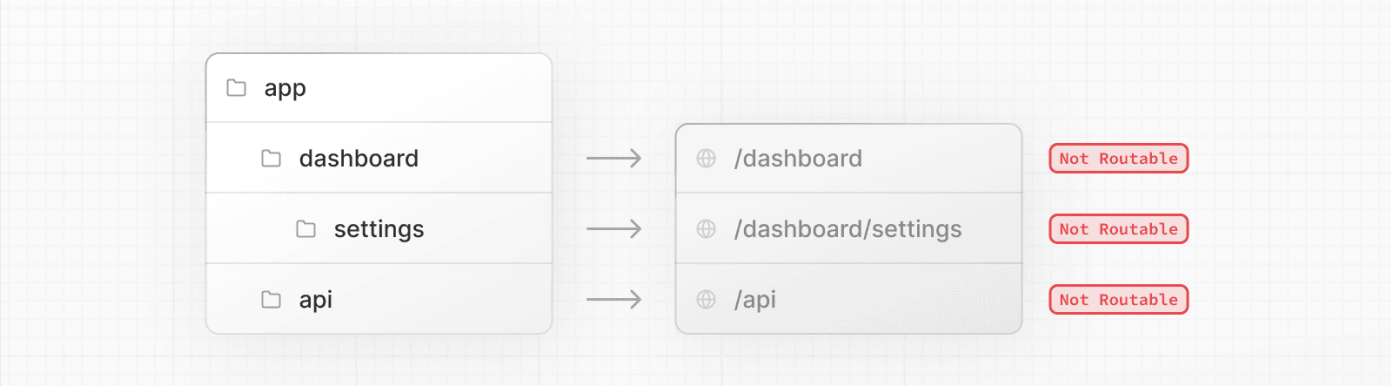
Routing Files

layout	.js .jsx .tsx	Layout
page	.js .jsx .tsx	Page
loading	.js .jsx .tsx	Loading UI
not-found	.js .jsx .tsx	Not found UI
error	.js .jsx .tsx	Error UI
global-error	.js .jsx .tsx	Global error UI
route	.js .ts	API endpoint
template	.js .jsx .tsx	Re-rendered layout
default	.js .jsx .tsx	Parallel route fallback page

https://nextjs.org/docs/app/getting-started/project-structure

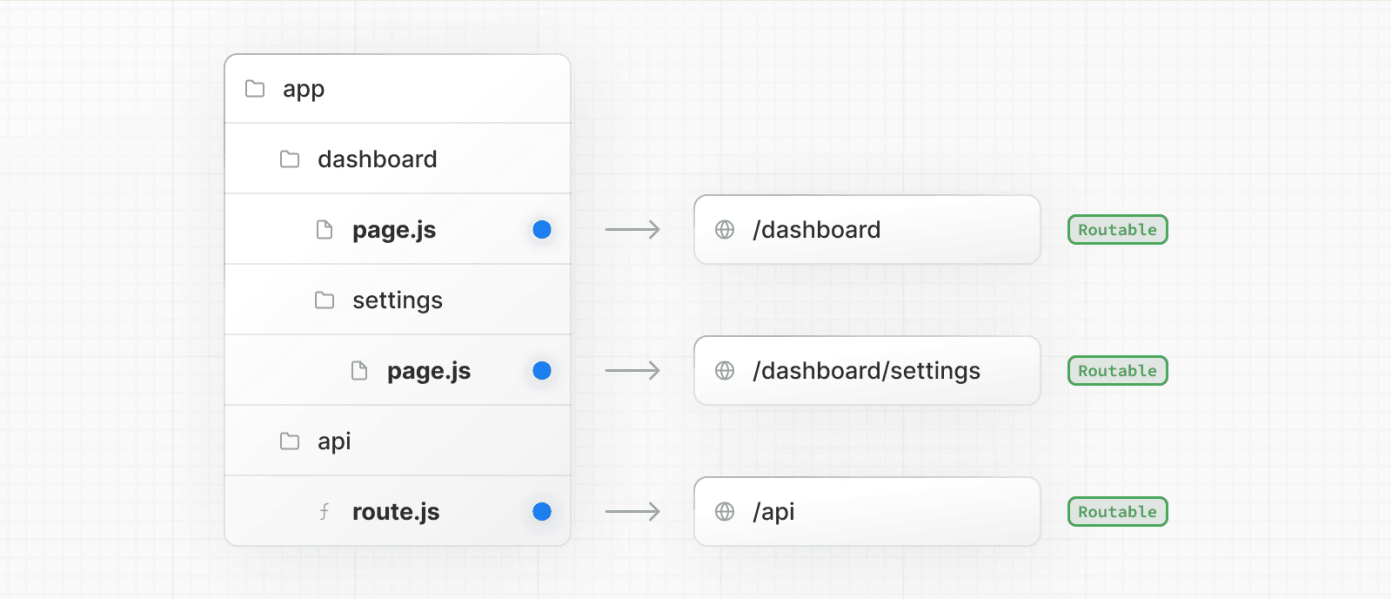
Route is defined by file structure

- Not public accessible



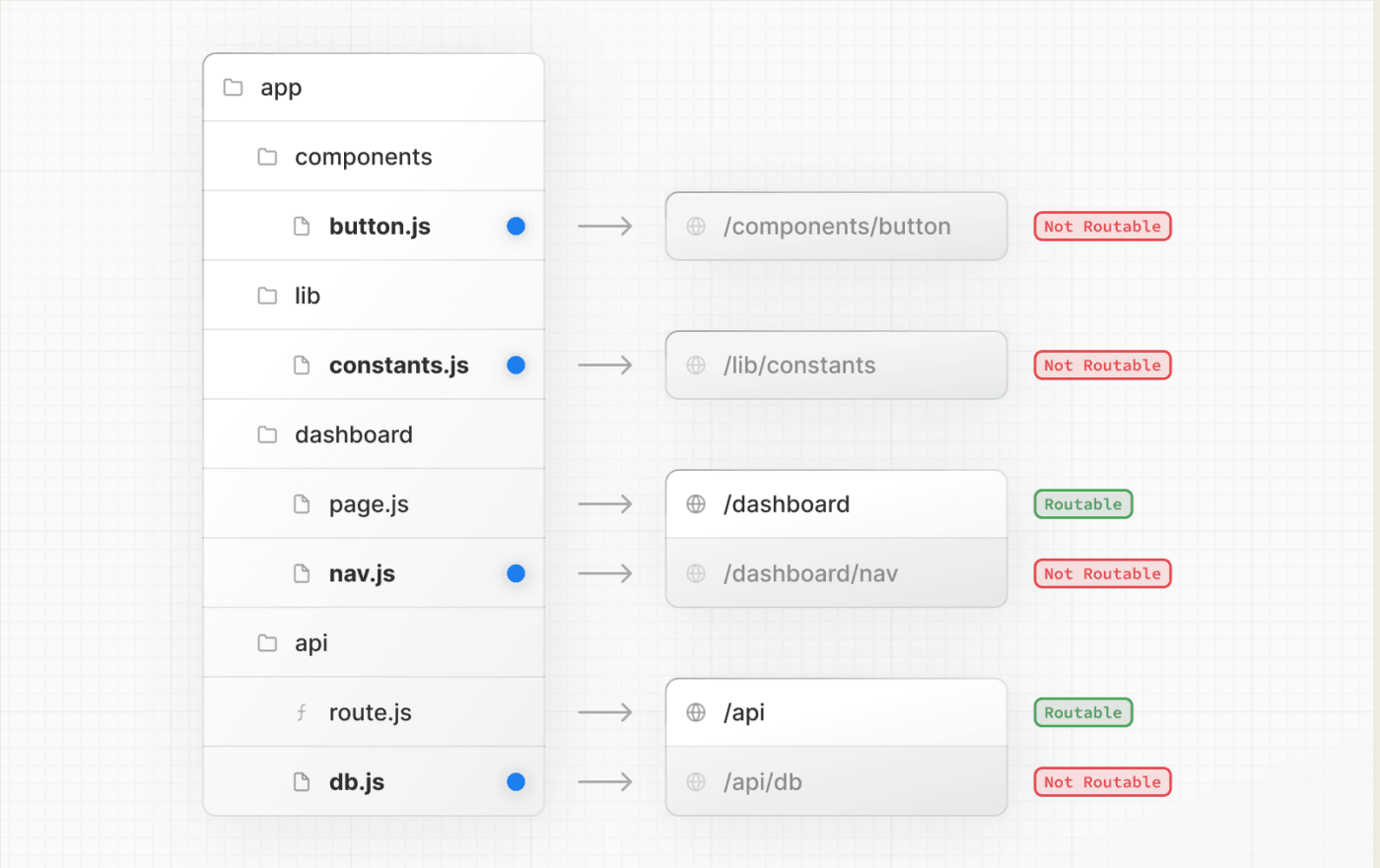
Route is defined by file structure

- publicly accessible
- Only the content returned by page.js [`page.ts`] or route.js [`route.ts`] is sent to the client



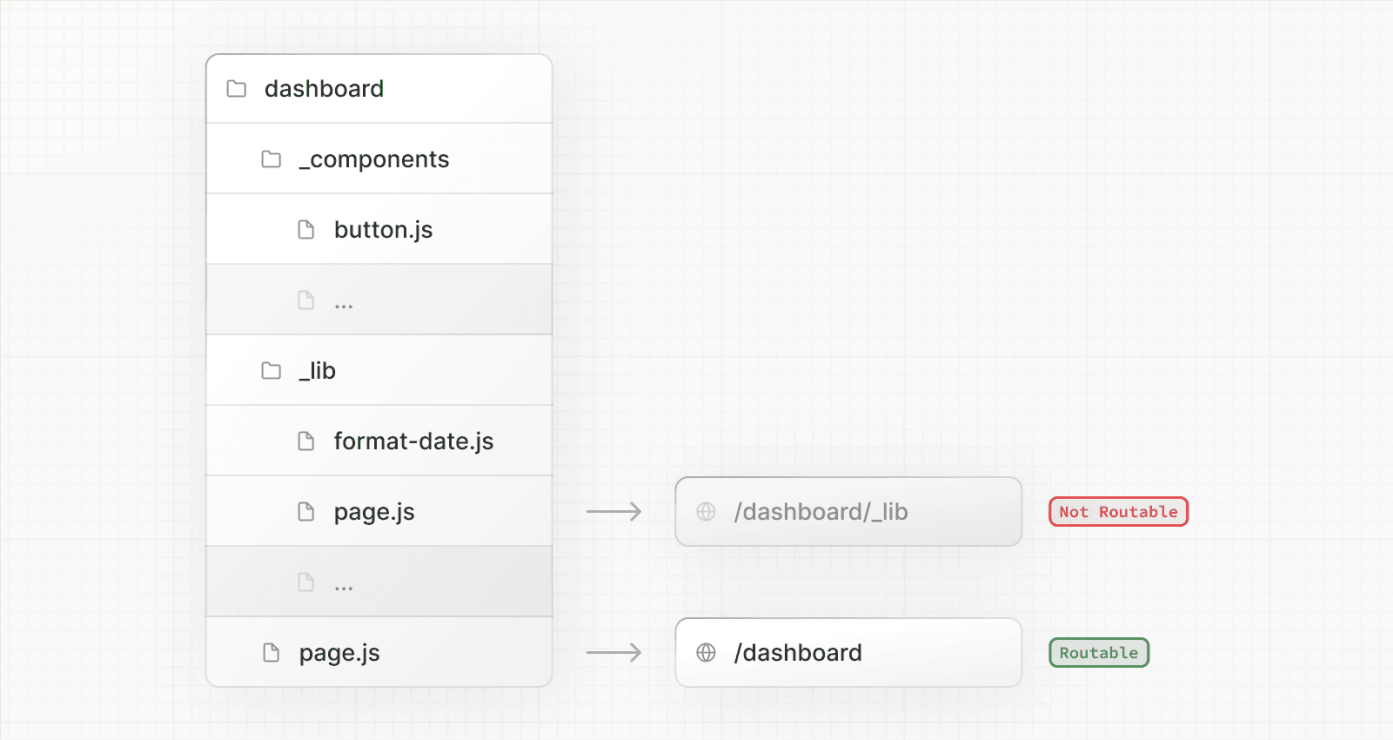
Route is defined by file structure with a caveat

- Project files can be put inside the app folder
- These files are not routable
- We can keep the files outside the app folder as well



Private folders (_foldername)

- prefixing a folder with an underscore



File-System based Routing

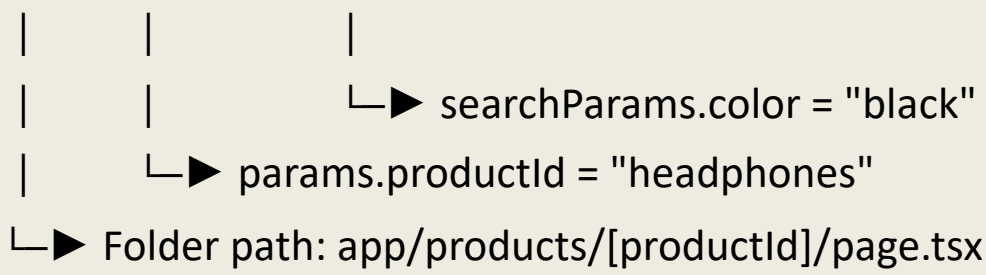
App Router uses nested folders to map routes directly to URLs.

- app/page.tsx maps to /
- app/about/page.tsx maps to /about
- routing files: layout.tsx, loading.tsx, error.tsx
- A route becomes public when a page or route file exists.

```
app/
├── page.tsx                    <-- Route: /
├── about/
│   └── page.tsx                <-- Route: /about
└── products/
    ├── page.tsx                <-- Route: /products
    └── [productId]/
        └── page.tsx            <-- Route: /products/:productId (Dynamic)
```

URL mapping

URL: https://mywebsite.com/products/headphones?color=black



Dynamic Routing

Use square brackets to create dynamic segment handlers

- app/blog/[slug]/page.tsx maps to /blog/post-1
  - when you want to match exactly one segment of a URL path.

```
app/
└── blog/
    └── [slug]/
        └── page.tsx   <-- Matches 1 dynamic segment after /blog/
```

Requested URL Path	Matches?	params Received	Notes
/blog	❌ No	—	Requires app/blog/page.tsx
/blog/post-1	✅ **Yes**	{ slug: 'post-1' }	Standard match
/blog/tech/nextjs	❌ No	—	Fails (too many segments)

Dynamic Routing

- [...slug] handles catch-all segments.
  - Use three dots inside square brackets to match **one or more nested path segments**. It captures all sub-paths into an array.

```
app/
└── docs/
    └── [...slug]/
        └── page.tsx   <-- Matches /docs/a, /docs/a/b, /docs/a/b/c, etc.
```

Requested URL Path	Matches?	params Received	Notes
/docs	❌ No	—	**Requires at least 1 segment**
/docs/setup	✅ **Yes**	{ slug: ['setup'] }	1 segment array
/docs/setup/install/mac	✅ **Yes**	{ slug: ['setup', 'install', 'mac'] }	3 segment array

Dynamic Routing

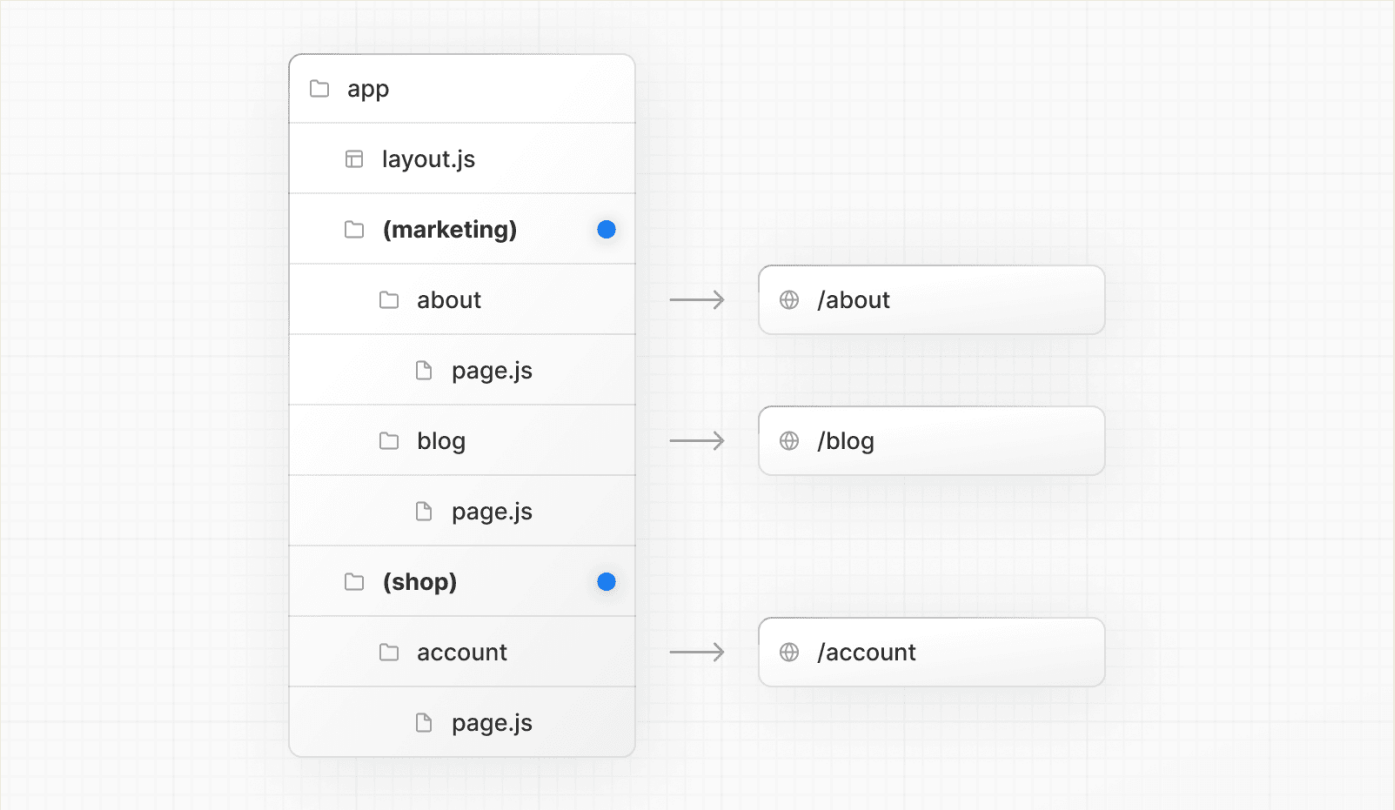
- [[...slug]] handles optional catch-all segments
  - Use double square brackets with three dots to match **zero or more path segments**. This allows a single page.tsx file to handle both the base route and any nested sub-routes.

```
app/
└── shop/
    └── [[...slug]]/
        └── page.tsx   <-- Matches /shop AND /shop/shoes AND /shop/shoes/nike, etc.
```

Requested URL Path	Matches?	params Received	Notes
/shop	✅ **Yes**	{ slug: undefined }	**Matches root folder with zero segments!**
/shop/shoes	✅ **Yes**	{ slug: ['shoes'] }	1 segment array
/shop/shoes/nike/red	✅ **Yes**	{ slug: ['shoes', 'nike', 'red'] }	3 segment array

Organise routes without affecting URL path

- Create a group to keep related routes together
- Use (foldername)



https://nextjs.org/docs/app/getting-started/project-structure

Nest.js App Router

- parentheses () and square brackets [] serve two fundamentally different routing purposes: organisation versus dynamic matching.

Feature	`(folder)` — Route Groups	`[param]` — Dynamic Routes
Purpose	Visual organization and layout scoping	Capturing dynamic path parameters
URL Impact	**Omitted** from the final URL path	**Included** as the variable segment in the URL
Data Passing	None	Passed as a `params` object to layouts/pages
Common Use	Distinct layouts (e.g., auth vs. dashboard)	Resource pages (e.g., blog posts, user IDs)

Navigation with Parameters (GET)

- Route Params
  - Accessed via page props. In Next.js App Router, params holds dynamic URL path segments like `id` or `slug`.

```
app/
└── users/
    └── [id]/
        └── page.tsx   <-- Handles /users/1, /users/42, /users/abc, etc.
```

When a user visits...	await params evaluates to...	Rendered Output
https://mywebsite.com/users/42	{ id: "42" }	"Currently viewing details for User ID: **42**"
https://mywebsite.com/users/john-doe	{ id: "john-doe" }	"Currently viewing details for User ID: **john-doe**"

Navigation with Parameters

- Search Params
  - Query parameters (e.g., ?page=2&sort=asc) are passed via searchParams prop or read using useSearchParams() hook (on client).

```
app/
└── products/
    └── page.tsx   <-- Handles URLs like /products?category=shoes&sort=asc
```

- **Server-side** (via the searchParams prop in a Server Component)
- **Client-side** (via the useSearchParams() hook in a Client Component)

Server-Side Example (searchParams Prop)

- In Next.js App Router Server Components, `searchParams` is passed as an asynchronous Promise to the page props.

```
// app/products/page.tsx

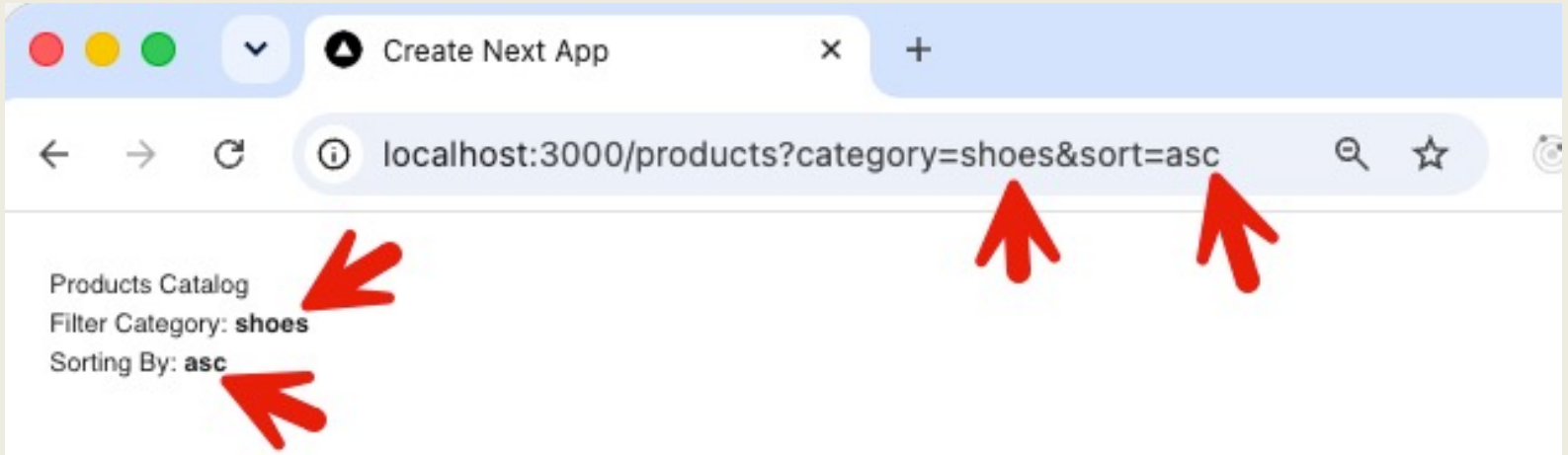
interface ProductsPageProps {
  searchParams: Promise<{
    category?: string; // Optional query string: ?category=shoes
    sort?: string;     // Optional query string: ?sort=asc
  }>;
}

export default async function ProductsPage({ searchParams }: ProductsPageProps) {
  // Await the searchParams Promise
  const { category = "all", sort = "featured" } = await searchParams;

  return (
    <main style={{ padding: "2rem", fontFamily: "sans-serif" }}>
      <h1>Products Catalog</h1>
      <p>Filter Category: <strong>{category}</strong></p>
      <p>Sorting By: <strong>{sort}</strong></p>
    </main>
  );
}
```

Server-Side Example (searchParams Prop)

- In Next.js App Router Server Components, `searchParams` is passed as an asynchronous Promise to the page props.



Client-Side Example (`useSearchParams` Hook)

- If you need to read query parameters inside an interactive client component (for example, to respond dynamically to input controls or button clicks), use the useSearchParams() hook from next/navigation.

```tsx
// app/products/SearchFilters.tsx
"use client";

import { useSearchParams } from "next/navigation";

export default function SearchFilters() {
  const searchParams = useSearchParams();

  // Read parameters using standard URLSearchParams methods (.get())
  const category = searchParams.get("category") ?? "all";
  const sort = searchParams.get("sort") ?? "featured";

  return (
    <div style={{ border: "1px solid #ccc", padding: "1rem", borderRadius: "8px" }}>
      <h3>Active Client Filters</h3>
      <p>Current Category: <strong>{category}</strong></p>
      <p>Current Sort Order: <strong>{sort}</strong></p>
    </div>
  );
}
```

Client-Side Example (useSearchParams Hook)

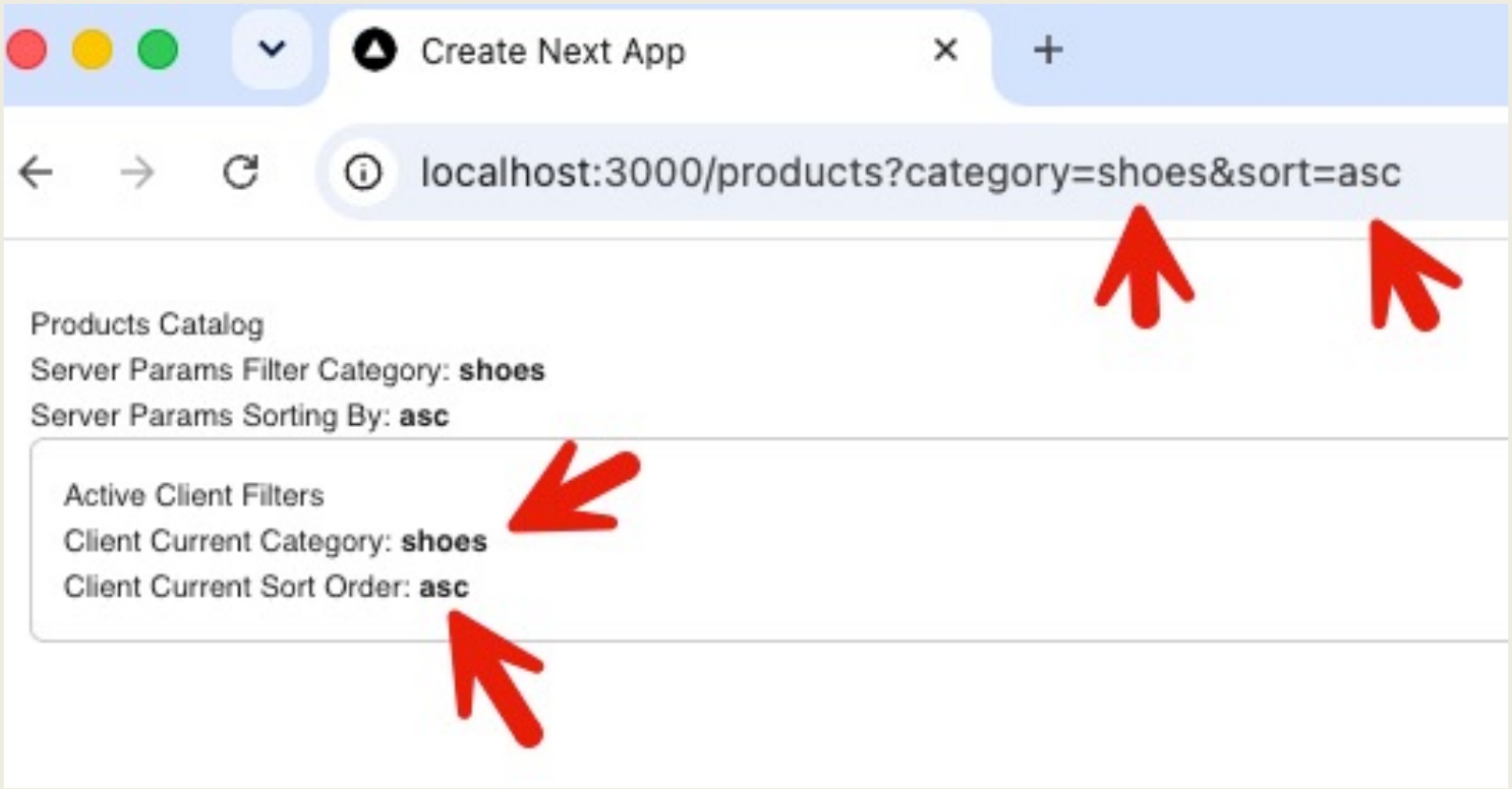
app > products > page.tsx > ProductsPage

```tsx
import SearchFilters from "./SearchFilters";
import { Suspense } from "react";

{/* Render the Client Component inside a Suspense boundary */}
<Suspense fallback={<div>Loading filters...</div>}>
  <SearchFilters />
</Suspense>
```

<Suspense> is a built-in wrapper component that acts as a **loading boundary**.
It tells React: *"While the components inside me are waiting for asynchronous work to finish (like fetching data or reading URL search parameters), don't freeze the whole page—show this fallback UI (spinner/skeleton) instead."*

Client-Side Example (useSearchParams Hook)



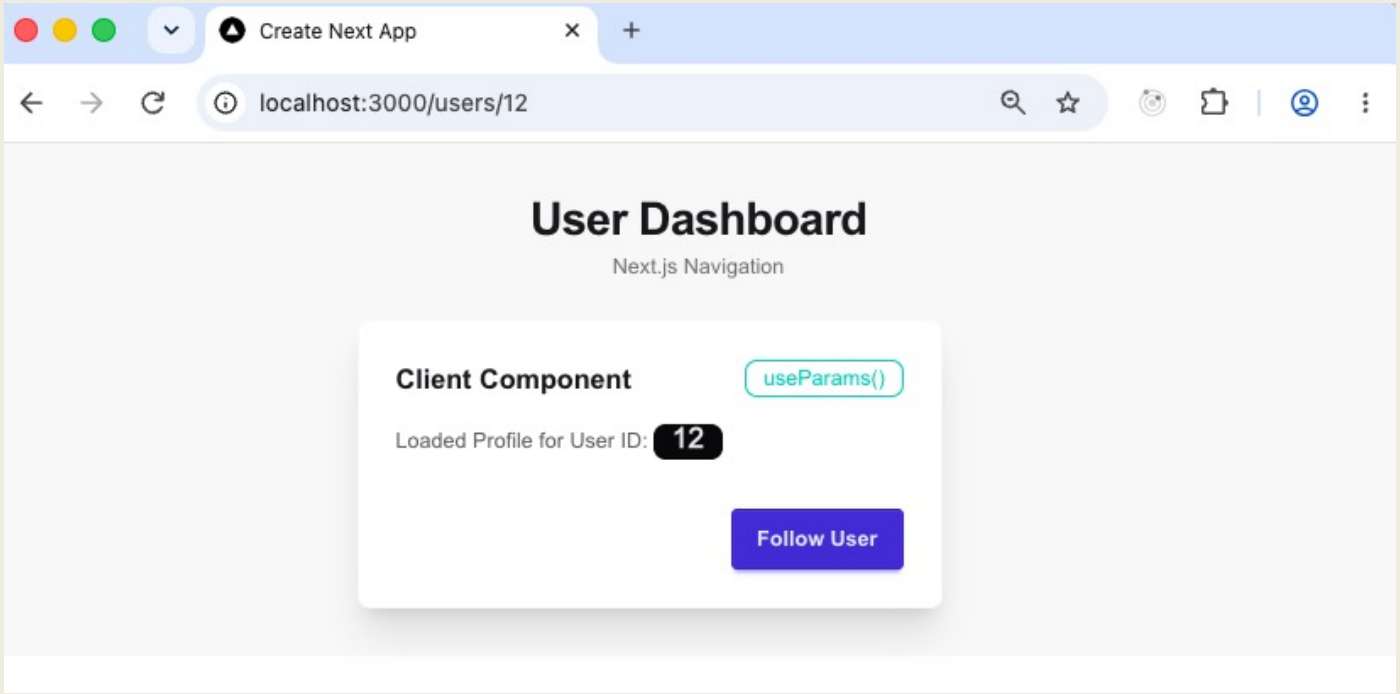
Navigation with Parameters

- Client Navigation
  - Use `useParams()` inside Client Components to read dynamic URL parameters directly from the router context.

```
app/
└── users/
    └── [userId]/
        └── page.tsx   <-- Route: /users/123 or /users/john-doe
```

Navigation with Parameters

- Client Navigation
  - Use `useParams()` inside Client Components to read dynamic URL parameters directly from the router context.



Navigation with Parameters

app > users > [userId] > UserProfileClient.tsx > UserProfileClient

```tsx
"use client";

import { useParams } from "next/navigation";
import { useState } from "react";

export default function UserProfileClient() {
  // 1. Read dynamic route params from context using TypeScript generics
  const params = useParams<{ userId: string }>();
  const userId = params.userId;

  const [isFollowing, setIsFollowing] = useState<boolean>(false);

  return (
    <div className="card w-96 bg-base-100 shadow-xl border border-base-200">
      <div className="card-body">
        <div className="flex justify-between items-center">
          <h2 className="card-title text-base-content">Client Component</h2>
          <span className="badge badge-accent badge-outline">useParams()</span>
        </div>

        <p className="py-2 text-sm text-base-content/70">
          Loaded Profile for User ID:{" "}
          <span className="badge badge-neutral font-mono font-semibold">
            {userId}
          </span>
        </p>

        <div className="card-actions justify-end mt-4">
          <button
            onClick={() => setIsFollowing(!isFollowing)}
            className={`btn ${
              isFollowing ? "btn-outline btn-error" : "btn-primary"
            }`}
          >
            {isFollowing ? "Unfollow" : "Follow User"}
          </button>
        </div>
      </div>
    </div>
  );
}
```

Client vs Server Components

- Next.js components are Server Components by default. Choose the right component model for performance:

- Server Components: Fetch data near database, reduce JS bundle size, zero client overhead.
- Client Components: Marked with 'use client' directive at file top.
- Interactivity: Client components handle useState, useEffect, and event listeners.

Data Fetching

Fetch, Axios API, ORM, Database

Client side fetching

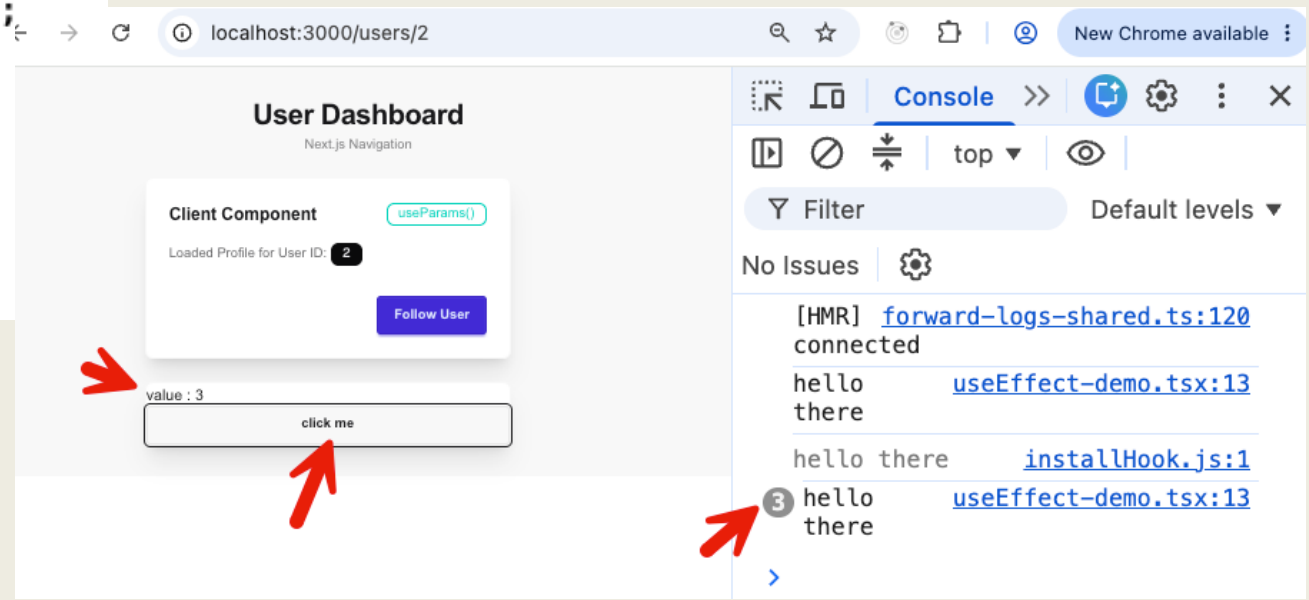
Scenario

- An app sends an HTTP request for data and displays the returned results.
- HTTP request (e.g. fetch API) – this is not controlled by REACT – “external”.
- Use `useEffect` hook to connect to ‘external’ system
- `useEffect` hook let us govern when and what to re-render.
- By default, `useEffect` hook triggers re-rendering when state value changes.
- We can configure it to run only on first render by passing an empty array as dependencies

useEffect hook

- By default, useEffect hook runs at every render.

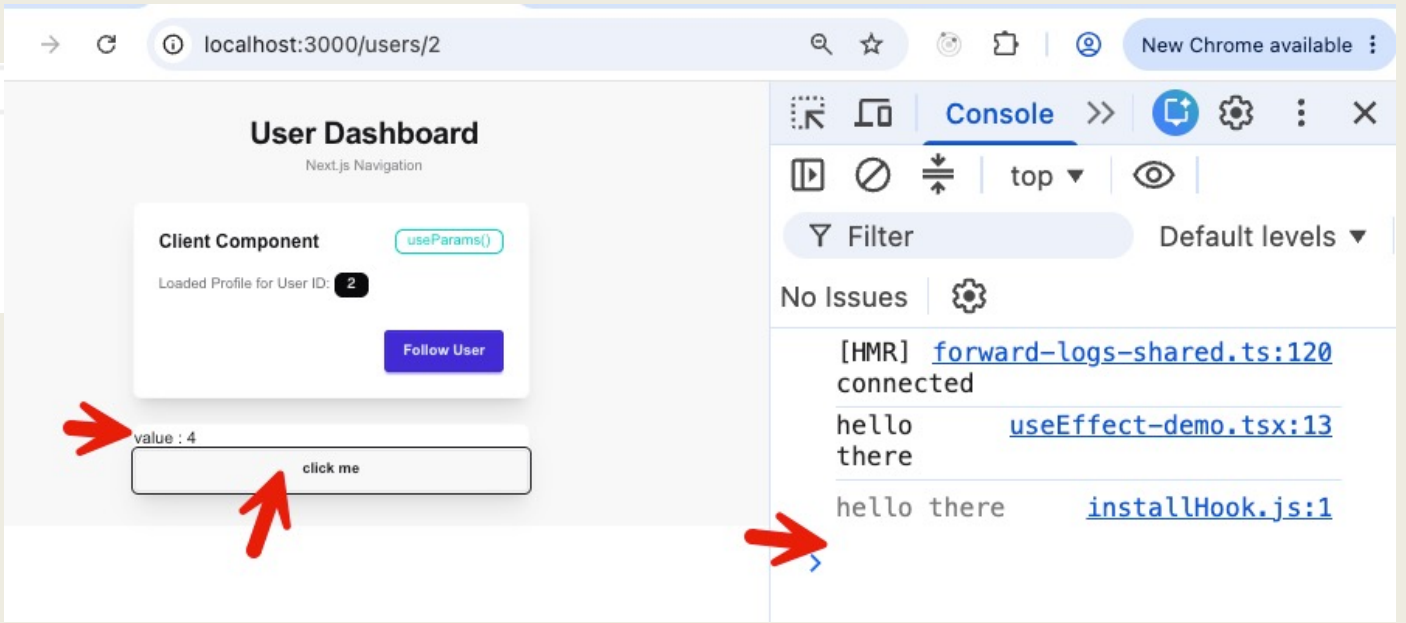
```
11
12  const sayHello = () => {
13    console.log("hello there");
14  };
15
16  useEffect(() => {
17    sayHello();
18  });
19
```



useEffect hook

- If an empty array is specified as the dependencies, then it will only run at first render, this makes it very useful for fetching data.

```
19
20  useEffect(() => {
21    sayHello();
22  }, []);
23
```



Fetch

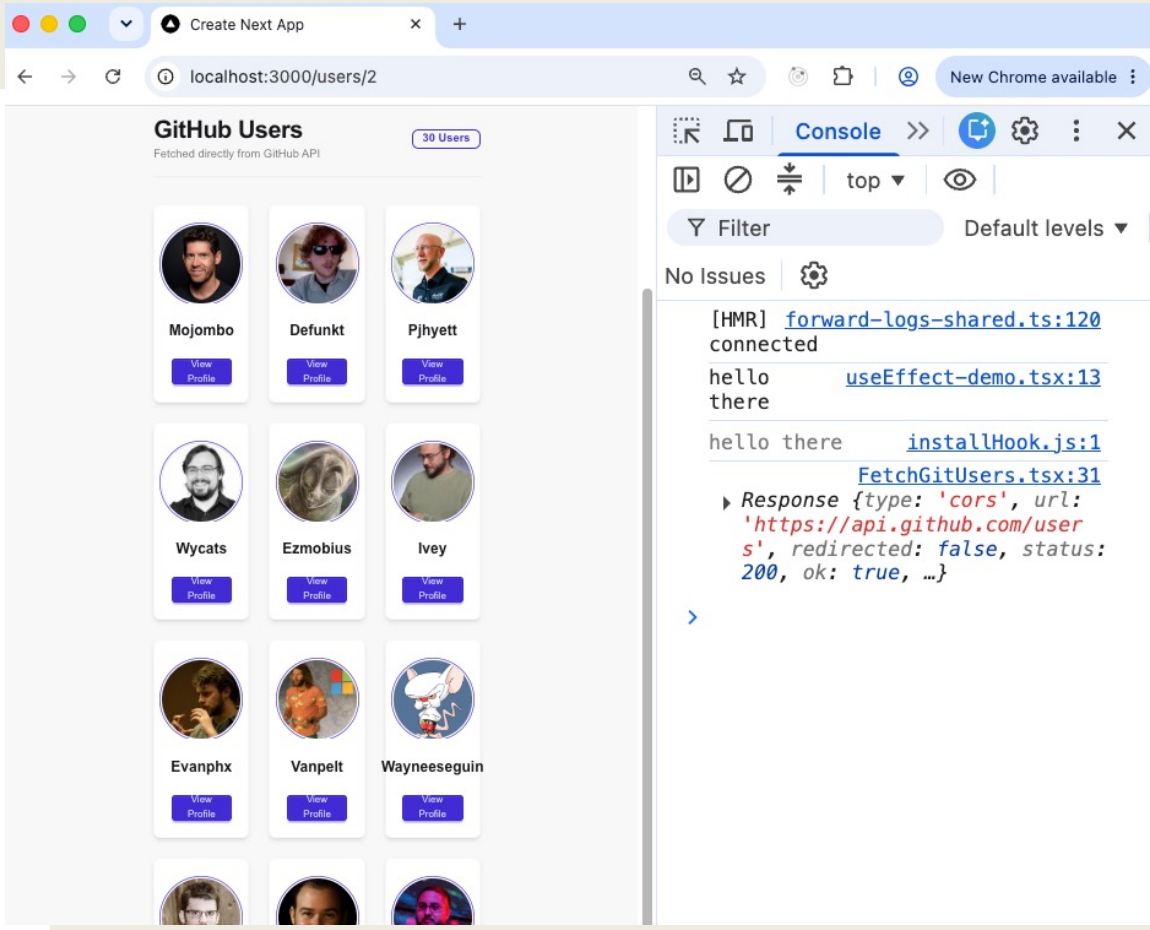
```
24
25    const fetchData = async () => {
26      try {
27        setLoading(true);
28        setError(null);
29
30        const response = await fetch(URL, { signal: controller.signal });
31
32        if (!response.ok) {
33          throw new Error(`HTTP Error! Status: ${response.status}`);
34        }
35
36        const data: GitHubUser[] = await response.json();
37        setUsers(data);
38      } catch (err: unknown) {
39        // Ignore abort errors caused by component unmounting
40        if (err instanceof Error && err.name === "AbortError") return;
41
42        if (err instanceof Error) {
43          setError(err.message);
44        } else {
45          setError("An unexpected error occurred.");
46        }
47      } finally {
48        setLoading(false);
49      }
50    };
51
```

Anything wrong with this code?

```
95
96  return (
97    <section className="container mx-auto p-6 max-w-5xl">
98      {/* Header */}
99      <div className="flex items-center justify-between border-b border-base-300 pb-4 mb-8">
100       <div>
101         <h2 className="text-3xl font-black text-base-content tracking-tight">
102           GitHub Users
103         </h2>
104         <p className="text-sm text-base-content/60 mt-1">
105           Fetched directly from GitHub API
106         </p>
107       </div>
108       <div className="badge badge-primary badge-outline font-semibold">
109         {users.length} Users
110       </div>
111     </div>
112
113     {/* Users Grid */}
114     <ul className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-6">
115       {users.map((user) => {
116         const { id, login, avatar_url, html_url } = user;
117         return (
118           <li key={id}>
119             {/* DaisyUI Card Component */}
120             <div className="card bg-base-100 shadow-md border border-base-200 hover:shadow-xl transition-all duration-200">
121               <figure className="px-6 pt-6">
122                 {/* DaisyUI Avatar Container */}
123                 <div className="avatar">
124                   <div className="w-24 rounded-full ring ring-primary ring-offset-base-100 ring-offset-2">
125                     <img src={avatar_url} alt={`${login}'s avatar`} />
126                   </div>
127                 </div>
128               </figure>
129               <div className="card-body items-center text-center p-5">
130                 <h3 className="card-title text-lg font-bold capitalize text-base-content">
131                   {login}
132                 </h3>
133                 <div className="card-actions mt-3 w-full">
134                   <a
135                     href={html_url}
136                     target="_blank"
137                     rel="noreferrer"
138                     className="btn btn-primary btn-sm btn-block font-medium"
139                   >
140                     View Profile
141                   </a>
142                 </div>
143               </div>
144             </div>
145           </li>
146         );
147       })}
148     </ul>
149   </section>
150 );
```

Fetch

```
20
21  useEffect(() => {
22    // 3. Native Web API AbortController for fetch cleanup
23    const controller = new AbortController();
24
25    const fetchData = async () => {
26      try {
27        setLoading(true);
28        setError(null);
29
30        const response = await fetch(URL, { signal: controller.signal });
31
32        if (!response.ok) {
33          throw new Error(`HTTP Error! Status: ${response.status}`);
34        }
35
36        const data: GitHubUser[] = await response.json();
37        setUsers(data);
38      } catch (err: unknown) {
39        // Ignore abort errors caused by component unmounting
40        if (err instanceof Error && err.name === "AbortError") return;
41
42        if (err instanceof Error) {
43          setError(err.message);
44        } else {
45          setError("An unexpected error occurred.");
46        }
47      } finally {
48        setLoading(false);
49      }
50    };
51
52    fetchData();
53
54    // 4. Clean up ongoing fetch if component unmounts
55    return () => {
56      controller.abort();
57    };
58  }, []);
59
```



AbortController

- AbortController is a **native Web API built directly into JavaScript runtime environments** (both modern web browsers and Node.js), just like setTimeout, console.log etc, it is available everywhere automatically
- It is useful for:
  - Preventing memory leaks & React state warning
  - Preventing "Race Conditions" (Stale Data Overwrites)
  - Saving Network Bandwidth & Server Resources

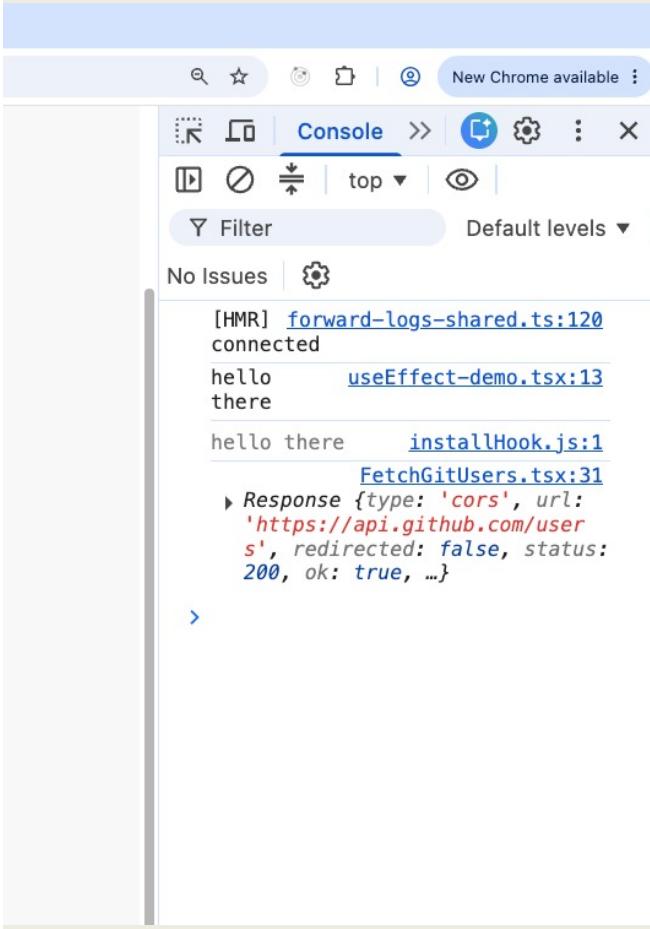
Fetch

```
20
21  useEffect(() => {
22    // 3. Native Web API AbortController for fetch cleanup
23    const controller = new AbortController();
24
25    const fetchData = async () => {
26      try {
27        setLoading(true);
28        setError(null);
29
30        const response = await fetch(URL, { signal: controll
31
32        if (!response.ok) {
33          throw new Error(`HTTP Error! Status: ${response.st
34        }
35
36        const data: GitHubUser[] = await response.json();
37        setUsers(data);
38      } catch (err: unknown) {
39        // Ignore abort errors caused by component unmountin
40        if (err instanceof Error && err.name === "AbortError
41
42        if (err instanceof Error) {
43          setError(err.message);
44        } else {
45          setError("An unexpected error occurred.");
46        }
47      } finally {
48        setLoading(false);
49      }
50    };
51
52    fetchData();
53
54    // 4. Clean up ongoing fetch if component unmounts
55    return () => {
56      controller.abort();
57    };
58  }, []);
59
```

FetchGitUsers.tsx:31

▼ Response {type: 'cors', url: 'https://api.github.com/users', redirected: false, status: 200, ok: true, …} i

- body: (...)
- bodyUsed: true
- ▼ headers: Headers
  - ▶ [[Prototype]]: Headers
- ok: true
- redirected: false
- status: 200
- statusText: ""
- type: "cors"
- url: "https://api.github.co
- ▼ [[Prototype]]: Response
  - ▶ arrayBuffer: ƒ arrayBuffe
  - ▶ blob: ƒ blob()
  - body: (...)
  - bodyUsed: (...)
  - ▶ bytes: ƒ bytes()
  - ▶ clone: ƒ clone()
  - ▶ formData: ƒ formData()
  - headers: (...)
  - ▶ json: ƒ json()
  - ok: (...)
  - redirected: (...)
  - status: (...)
  - statusText: (...)
  - ▶ text: ƒ text()



Image

https://nextjs.org/docs/app/api-reference/components/image

- The Next.js Image component extends the HTML <img> element for automatic image optimisation.

```jsx
{/* <img src={avatar_url} alt={`${login}'s avatar`} /> */}
<Image
  src={avatar_url}
  alt={`${login}'s avatar`}
  width={600}
  height={600}
/>
```

TS next.config.ts > ...

```ts
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  /* config options here */
  images: {
    remotePatterns: [
      {
        protocol: "https",
        hostname: "avatars.githubusercontent.com",
        port: "",
        pathname: "/**",
      },
    ],
  },
};

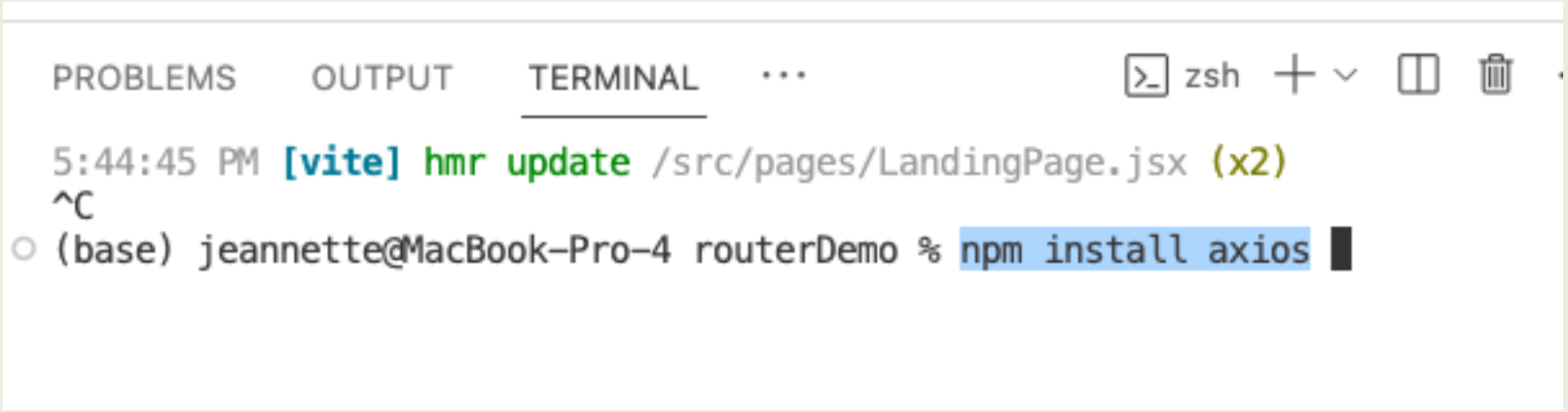
export default nextConfig;
```

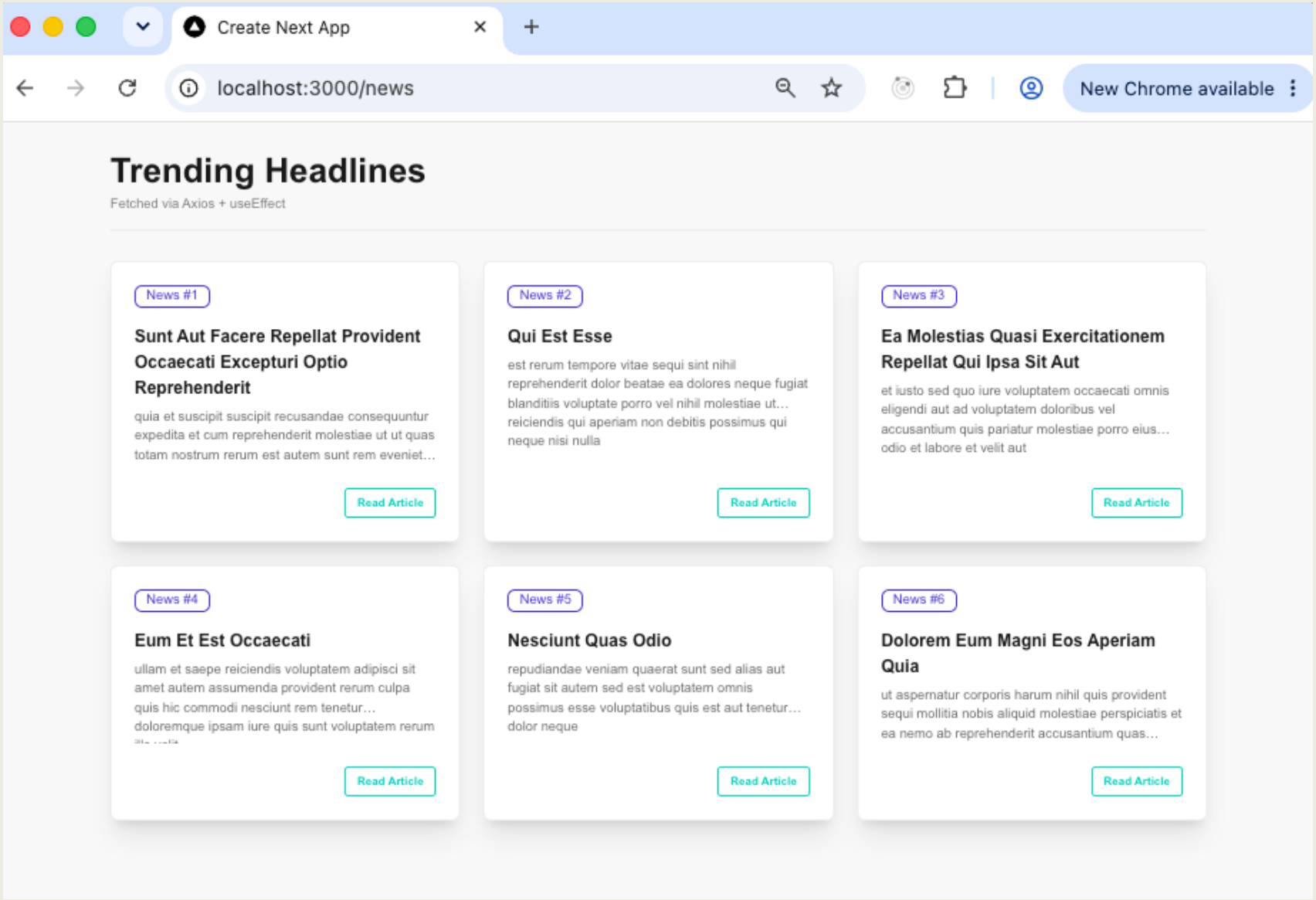
Fetch or Axios?

- Fetch is very useful API, it works straight out of the box. It supports both `async await` and ES6 Asynchronous `.then()`
- Axios API does the same job, it's very popular for REACT. It also supports both `async await` and ES6 Asynchronous `.then()`, in addition, it provides useful error messages as well.

Install Axios API

- npm install axios




Create Next App
localhost:3000/news
New Chrome available
Trending Headlines
Fetched via Axios + useEffect
News #1
Sunt Aut Facere Repellat Provident Occaecati Excepturi Optio Reprehenderit
quia et suscipit suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam nostrum rerum est autem sunt rem eveniet…
Read Article
News #2
Qui Est Esse
est rerum tempore vitae sequi sint nihil reprehenderit dolor beatae ea dolores neque fugiat blanditiis voluptate porro vel nihil molestiae ut… reiciendis qui aperiam non debitis possimus qui neque nisi nulla
Read Article
News #3
Ea Molestias Quasi Exercitationem Repellat Qui Ipsa Sit Aut
et iusto sed quo iure voluptatem occaecati omnis eligendi aut ad voluptatem doloribus vel accusantium quis pariatur molestiae porro eius… odio et labore et velit aut
Read Article
News #4
Eum Et Est Occaecati
ullam et saepe reiciendis voluptatem adipisci sit amet autem assumenda provident rerum culpa quis hic commodi nesciunt rem tenetur… doloremque ipsam iure quis sunt voluptatem rerum
Read Article
News #5
Nesciunt Quas Odio
repudiandae veniam quaerat sunt sed alias aut fugiat sit autem sed est voluptatem omnis possimus esse voluptatibus quis est aut tenetur… dolor neque
Read Article
News #6
Dolorem Eum Magni Eos Aperiam Quia
ut aspernatur corporis harum nihil quis provident sequi mollitia nobis aliquid molestiae perspiciatis et ea nemo ab reprehenderit accusantium quas…
Read Article

use Axios API

- We need a component state (articles)
- Send an Axios HTTP GET request and fetch the data
  - Axios – results are stored in response.data
- Update the state value to the returned data

```tsx
"use client";

import { useState, useEffect } from "react";
import axios from "axios";

// 1. Define the TypeScript interface for a News Article item
interface Article {
  id: number;
  title: string;
  body: string;
}

export default function NewsPage() {
  // 2. Setup state for data, loading, and error handling
  const [articles, setArticles] = useState<Article[]>([]);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);

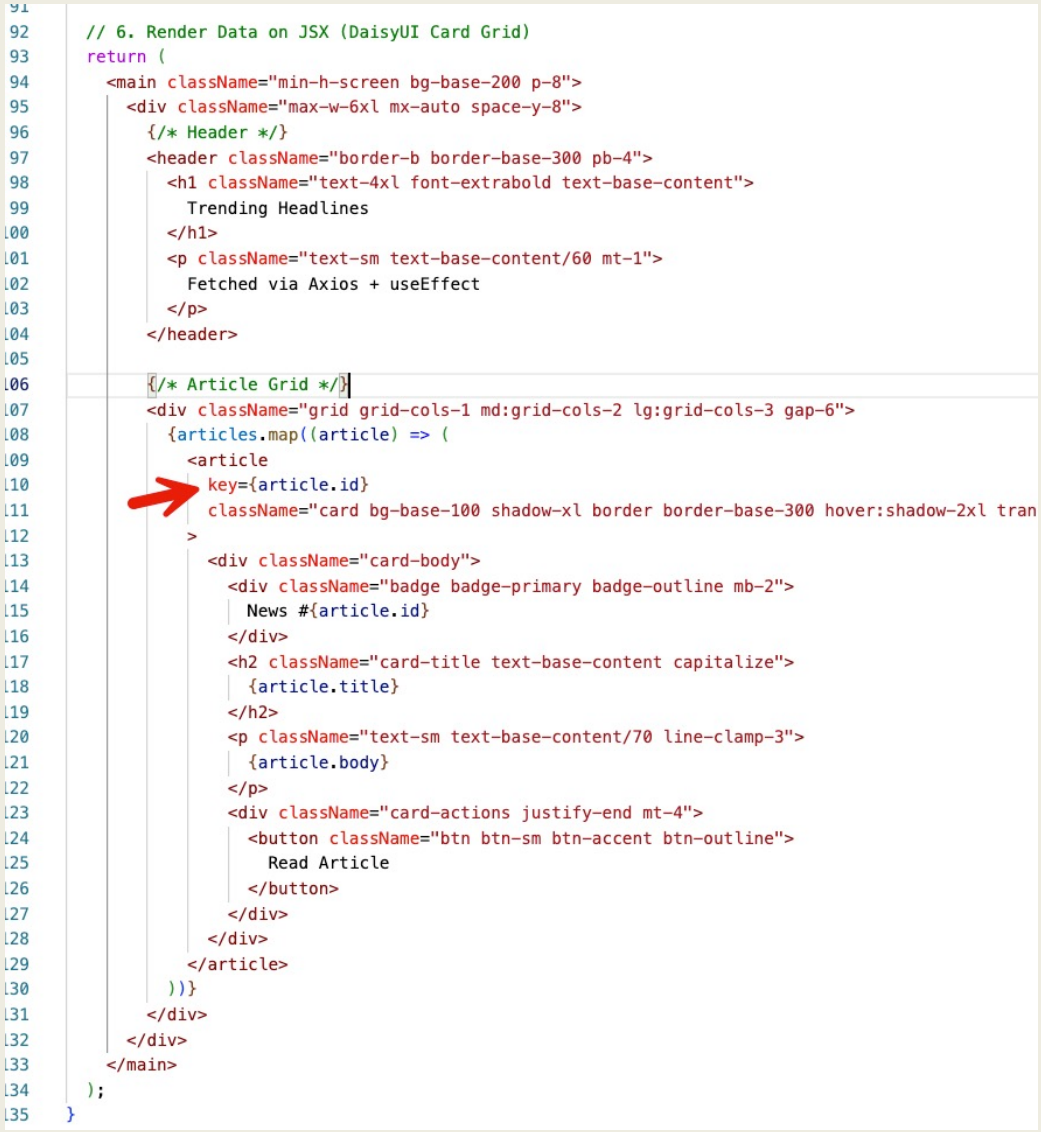
```

```
18
19  // 3. Fetch data inside useEffect
20  useEffect(() => {
21    // Create an AbortController to cleanup on component unmount
22    const controller = new AbortController();
23
24    const fetchNews = async () => {
25      try {
26        setLoading(true);
27        setError(null);
28
29        // Fetch top 6 mock news articles using Axios
30        const response = await axios.get<Article[]>(
31          "https://jsonplaceholder.typicode.com/posts?_limit=6",
32          { signal: controller.signal },
33        );
34
35        setArticles(response.data);
36      } catch (err: unknown) {
37        // Ignore errors caused by unmounting (aborted requests)
38        if (axios.isCancel(err)) return;
39
40        if (axios.isAxiosError(err)) {
41          setError(err.message || "Failed to fetch news articles.");
42        } else {
43          setError("An unexpected error occurred.");
44        }
45      } finally {
46        setLoading(false);
47      }
48    };
49
50    fetchNews();
51
52    // Cleanup function runs if the component unmounts
53    return () => {
54      controller.abort();
55    };
56  }, []); // Empty dependency array = runs ONCE when component mounts
57
```

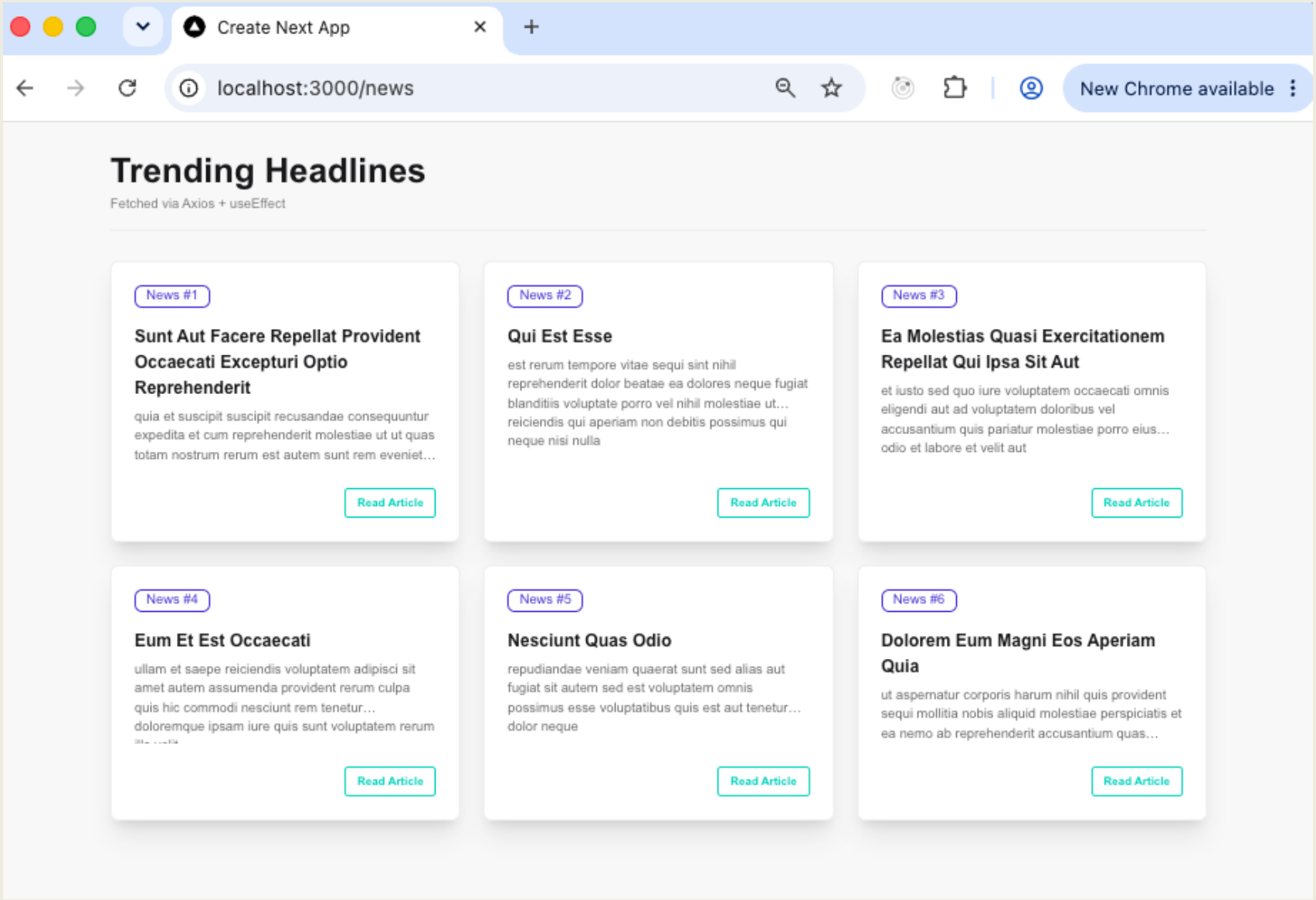
```
57
58  // 4. Render Loading State (DaisyUI Loading Spinner)
59  if (loading) {
60    return (
61      <main className="min-h-screen bg-base-200 flex flex-col items-center justify-center p-8">
62        <span className="loading loading-spinner loading-lg text-primary"></span>
63        <p className="mt-4 text-base-content/70">Fetching latest news...</p>
64      </main>
65    );
66  }
67
68  // 5. Render Error State (DaisyUI Alert)
69  if (error) {
70    return (
71      <main className="min-h-screen bg-base-200 p-8 flex justify-center items-center">
72        <div className="alert alert-error max-w-md shadow-lg">
73          <svg
74            xmlns="http://www.w3.org/2000/svg"
75            className="stroke-current shrink-0 h-6 w-6"
76            fill="none"
77            viewBox="0 0 24 24"
78          >
79            <path
80              strokeLinecap="round"
81              strokeLinejoin="round"
82              strokeWidth="2"
83              d="M10 14l2-2m0 0l2-2m-2 2l-2-2m2 2l2 2m7-2a9 9 0 11-18 0 9 9 9 0 0118 0z"
84            />
85          </svg>
86          <span>{error}</span>
87        </div>
88      </main>
89    );
90  }
91
```

Key prop

- Every child element requires a unique key prop
- We can use the ‘id’ provided by the data



```
91
92    // 6. Render Data on JSX (DaisyUI Card Grid)
93    return (
94      <main className="min-h-screen bg-base-200 p-8">
95        <div className="max-w-6xl mx-auto space-y-8">
96          {/* Header */}
97          <header className="border-b border-base-300 pb-4">
98            <h1 className="text-4xl font-extrabold text-base-content">
99              Trending Headlines
100           </h1>
101           <p className="text-sm text-base-content/60 mt-1">
102             Fetched via Axios + useEffect
103           </p>
104         </header>
105
106         {/* Article Grid */}
107         <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
108           {articles.map((article) => (
109             <article
110               key={article.id}
111               className="card bg-base-100 shadow-xl border border-base-300 hover:shadow-2xl tran
112             >
113               <div className="card-body">
114                 <div className="badge badge-primary badge-outline mb-2">
115                   News #{article.id}
116                 </div>
117                 <h2 className="card-title text-base-content capitalize">
118                   {article.title}
119                 </h2>
120                 <p className="text-sm text-base-content/70 line-clamp-3">
121                   {article.body}
122                 </p>
123                 <div className="card-actions justify-end mt-4">
124                   <button className="btn btn-sm btn-accent btn-outline">
125                     Read Article
126                   </button>
127                 </div>
128               </div>
129             </article>
130           ))}
131         </div>
132       </div>
133     </main>
134   );
135 }
```


Create Next App
localhost:3000/news
New Chrome available
Trending Headlines
Fetched via Axios + useEffect
News #1
Sunt Aut Facere Repellat Provident Occaecati Excepturi Optio Reprehenderit
quia et suscipit suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam nostrum rerum est autem sunt rem eveniet…
Read Article
News #2
Qui Est Esse
est rerum tempore vitae sequi sint nihil reprehenderit dolor beatae ea dolores neque fugiat blanditiis voluptate porro vel nihil molestiae ut… reiciendis qui aperiam non debitis possimus qui neque nisi nulla
Read Article
News #3
Ea Molestias Quasi Exercitationem Repellat Qui Ipsa Sit Aut
et iusto sed quo iure voluptatem occaecati omnis eligendi aut ad voluptatem doloribus vel accusantium quis pariatur molestiae porro eius… odio et labore et velit aut
Read Article
News #4
Eum Et Est Occaecati
ullam et saepe reiciendis voluptatem adipisci sit amet autem assumenda provident rerum culpa quis hic commodi nesciunt rem tenetur… doloremque ipsam iure quis sunt voluptatem rerum
Read Article
News #5
Nesciunt Quas Odio
repudiandae veniam quaerat sunt sed alias aut fugiat sit autem sed est voluptatem omnis possimus esse voluptatibus quis est aut tenetur… dolor neque
Read Article
News #6
Dolorem Eum Magni Eos Aperiam Quia
ut aspernatur corporis harum nihil quis provident sequi mollitia nobis aliquid molestiae perspiciatis et ea nemo ab reprehenderit accusantium quas…
Read Article

More axios

- By default, the request is GET
- It supports other HTTP requests too – POST, PATCH, PUT, DELETE etc
- Details see https://axios-http.com/docs/intro

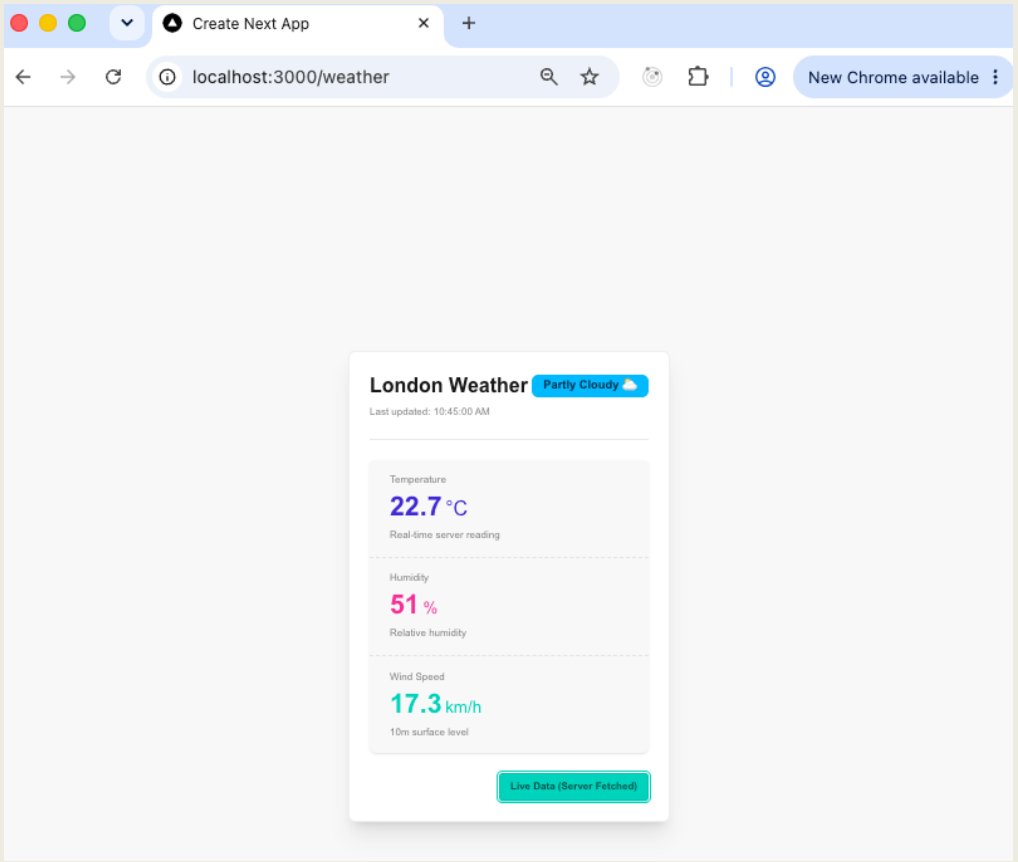
Server-Side Fetching

Server vs. Client Data Fetching

Contrast between client-side `useEffect` and `async` React Server Components (RSC).

- Client-side pattern
  - Requires 'use client', `useState`, and `useEffect`. Causes client waterfall network requests, loading flashes, and exposes API endpoints to the browser.
- Server Component Pattern
  - Turn the component into an `async` function and direct `await` fetch() inside the render tree. Zero client JS bundle overhead and secure secret key usage.

Real time weather info fetching



Real time weather info fetching

app > weather > page.tsx > WeatherPage

```tsx
import { JSX } from "react";

// 1. Define explicit TypeScript interfaces for the Open-Meteo response
interface OpenMeteoResponse {
  current: {
    time: string;
    temperature_2m: number;
    relative_humidity_2m: number;
    wind_speed_10m: number;
    weather_code: number;
  };
  current_units: {
    temperature_2m: string;
    relative_humidity_2m: string;
    wind_speed_10m: string;
  };
}

```

Real time weather info fetching

```
19  // Map WMO Weather Interpretation Codes to friendly labels & badge styles
20  // https://emojipedia.org/ copy and paste!
21  function getWeatherInfo(code: number): { label: string; badgeClass: string } {
22    if (code === 0) return { label: "Clear Sky ☀️", badgeClass: "badge-warning" };
23    if (code >= 1 && code <= 3)
24      return { label: "Partly Cloudy ⛅", badgeClass: "badge-info" };
25    if (code >= 45 && code <= 48)
26      return { label: "Foggy 🌫️", badgeClass: "badge-ghost" };
27    if (code >= 51 && code <= 67)
28      return { label: "Rain / Drizzle 🌧️", badgeClass: "badge-primary" };
29    if (code >= 71 && code <= 77)
30      return { label: "Snow ❄️", badgeClass: "badge-secondary" };
31    return { label: "Stormy ⛈️", badgeClass: "badge-error" };
32  }
```

Real time weather info fetching

```
32
33  // 2. Async Server Component - direct data fetching on the server
34  export default async function WeatherPage(): Promise<JSX.Element> {
35    // Fetching London forecast from Open-Meteo (No API key needed)
36    const res = await fetch(
37      "https://api.open-meteo.com/v1/forecast?latitude=51.5074&longitude=-0.1278&curre
38      {
39        // Revalidate every 10 minutes (ISR - Incremental Static Regeneration)
40        next: { revalidate: 600 },
41      },
42    );
43
44    if (!res.ok) {
45      throw new Error("Failed to fetch weather data from Open-Meteo.");
46    }
47
48    const data: OpenMeteoResponse = await res.json();
49    const weather = getWeatherInfo(data.current.weather_code);
50
```

Incremental Static Regeneration (ISR)

Stale-While-Revalidate Behaviour:

- When 600 seconds (10 minutes) pass, Next.js does not make the user wait for a new fetch on the next page load.
- Instead, it instantly serves the stale (older) cached version to that user, and triggers a background fetch/re-render.
- Only the next user (or the same user on their next refresh) gets the updated page.
- If 1,000 visitors hit your site during those 10 minutes, Open-Meteo is only called once.
- Next.js serves the pre-rendered static HTML to all 1,000 users straight from the cache.
- The re-fetching and re-rendering happen entirely on the server. The user's browser isn't running any client-side timers or JavaScript setInterval loops.

Incremental Static Regeneration (ISR)

- [Request 1 (0:00)] ----> Returns Cached Static HTML (Instant)
  - ↳ Next.js checks: Has 600 seconds passed? No.
- [Request 2 (5:00)] ----> Returns Cached Static HTML (Instant)
  - ↳ Next.js checks: Has 600 seconds passed? No.
- [Request 3 (10:05)] ---> Returns Stale Cached HTML to User (Instant!)
  - ↳ Next.js checks: Has 600 seconds passed? YES!
  - ↳ Background task: Triggers Open-Meteo fetch + Re-renders page
  - ↳ Saves new HTML & data to cache
- [Request 4 (10:10)] ---> Returns NEW Cached HTML to User!

On-demand revalidation

- allows you to purge cached data and re-render a page immediately when something changes (like a database update or user action), rather than waiting for a timer to expire.
- Next.js provides two functions in next/cache for this: **revalidatePath** and **revalidateTag**.

Using `revalidatePath`

- Code an action (server)
- Code a refresh button (client)
- Embed the refresh button to the page!

app > TS actions.ts > ...

```
1   "use server";
2
3   import { revalidatePath } from "next/cache";
4
5   export async function refreshWeatherData() {
6     // Perform your mutation or database operation here
7
8     // Revalidate the specific path
9     revalidatePath("/weather");
10  }
11
```

Using `revalidat`

- Code an action (server)
- Code a refresh button (client)
- Embed the refresh button to the page!

app > weather > RefreshButton.tsx > ...

```tsx
"use client";

import { refreshWeatherData } from "@/app/actions";
import { useTransition, JSX } from "react";

export default function RefreshButton(): JSX.Element {
  const [isPending, startTransition] = useTransition();

  return (
    <button
      className="btn btn-primary"
      disabled={isPending}
      onClick={() => {
        startTransition(async () => {
          await refreshWeatherData();
        });
      }}
    >
      {isPending ? "Refreshing..." : "Refresh Weather Now"}
    </button>
  );
}
```

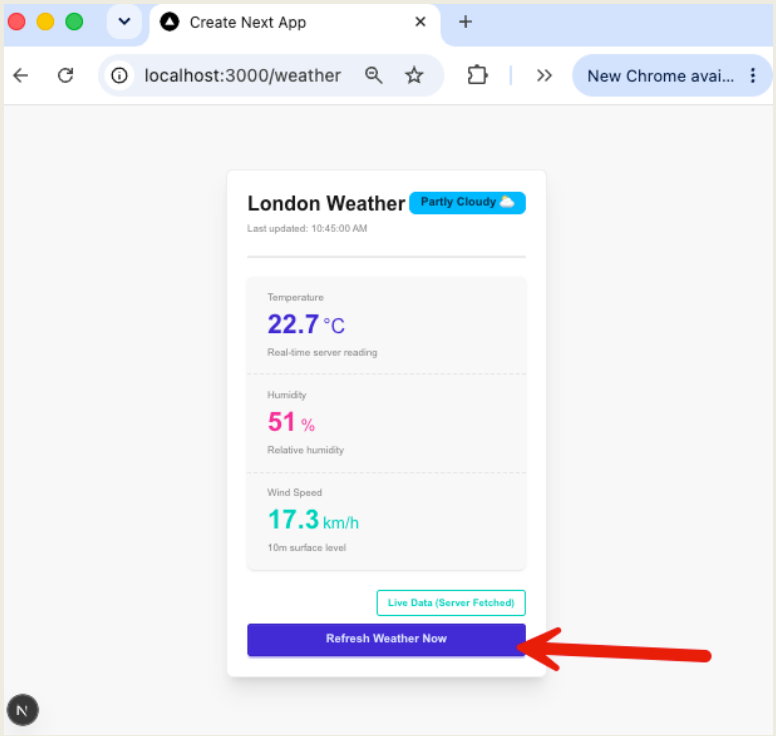
Using `revalidat`

app > weather > page.tsx > WeatherPage

```
1  import { JSX } from "react";
2  import RefreshButton from "./RefreshButton";
3
```

- Code an action (server)
- Code a refresh button (client)
- Embed the refresh button to the page!

```
117          </div>
118          <RefreshButton />
119        </div>
```



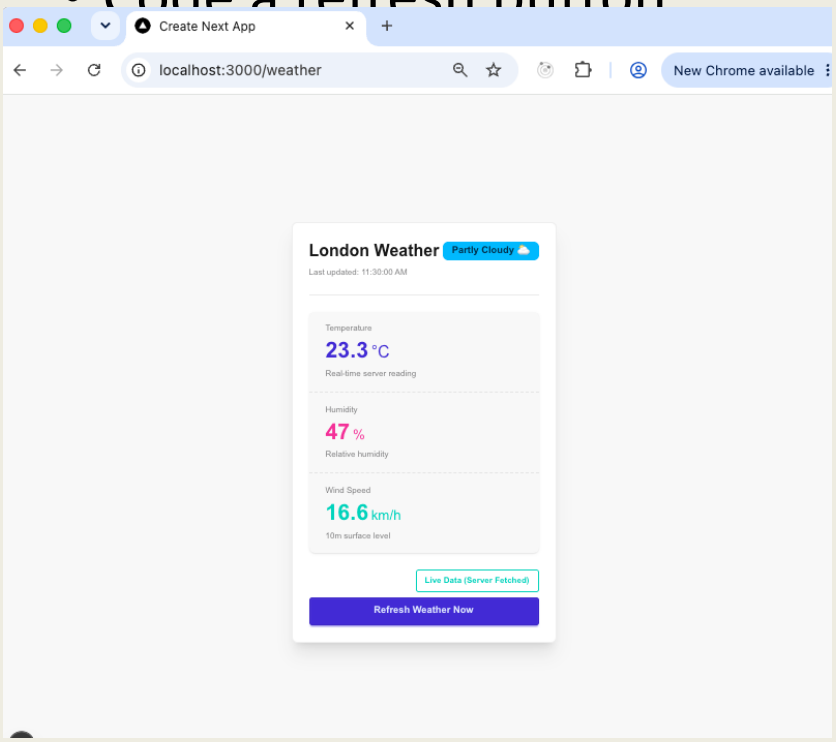
Using `revalidat`

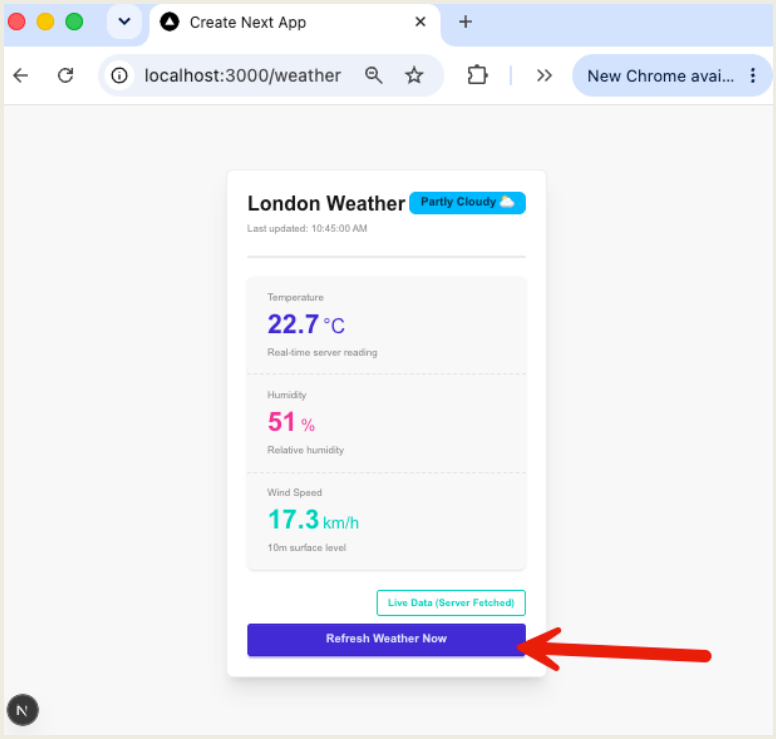
- Code an action (server)
- Code a refresh button

app > weather > page.tsx > WeatherPage

```
1   import { JSX } from "react";
2   import RefreshButton from "./RefreshButton";
3
```

```
117             </div>
118             <RefreshButton />
119         </div>
```





Using `revalidate`

- `revalidateTag` lets you tag specific fetch requests and revalidate all data associated with that tag across multiple pages at once.

Steps:

- Update tag
- And `revalidateTag`

```
35  // 2. Async Server Component - direct data fetching on the server
36  export default async function WeatherPage(): Promise<JSX.Element> {
37    // Fetching London forecast from Open-Meteo (No API key needed)
38    const res = await fetch(
39      "https://api.open-meteo.com/v1/forecast?latitude=51.5074&longitude=-0.1278&
40      {
41        // Revalidate every 10 minutes (ISR - Incremental Static Regeneration)
42        //   next: { revalidate: 600 },
43        next: {
44          tags: ["weather-data"], // <--- Label this request with a tag
45        },
46      },
47    );
```

app > TS actions.ts > ...

```
1  "use server";
2
3  import { revalidatePath, revalidateTag } from "next/cache";
4
5  export async function refreshWeatherData() {
6    // Perform your mutation or database operation here
7
8    // Revalidate the specific path
9    revalidatePath("/weather");
10   revalidateTag("weather-data", "max");
11 }
12
```

x-next-cache

- HIT is passive: Next.js knows the data is up-to-date, hands it over immediately, and performs zero work behind the scenes.
- STALE is active: Next.js knows the data is out of date, but rather than forcing the visitor to wait for an external API call , it hands them the old data instantly while silently fetching fresh data in the background

Value	Meaning	What the Server Does
`HIT`	Cache is valid and fresh.	Serves the cached response immediately. No backend work.
`MISS`	Not found in cache.	Executes the function/fetch, renders the response, serves it, and writes it to the cache for next time.
`STALE`	Tag invalidated (e.g. via `revalidateTag`), but stale data is permitted.	Serves the existing cached data instantly so the visitor experiences zero delay, while simultaneously kicking off a background refresh to populate new data.

Run Production-mode Server

```Bash
npm run build && npm run start
```

Test it yourself!

① **Prime the Cache**

Browser Tab 1

Visit your weather page at `http://localhost:3000/weather`. Check DevTools **Network** (`Disable cache` checked).
Verification: `x-next-cache` shows **`HIT`**.

② **Trigger Invalidation**

Browser Tab 2

Open a new tab and navigate to:
`http://localhost:3000/api/revalidate`
Verification: You see `{"revalidated": true}`. The tag `"weather-data"` is now marked stale.

Test it yourself!

③ **Inspect the Stale Response**

Browser Tab 1

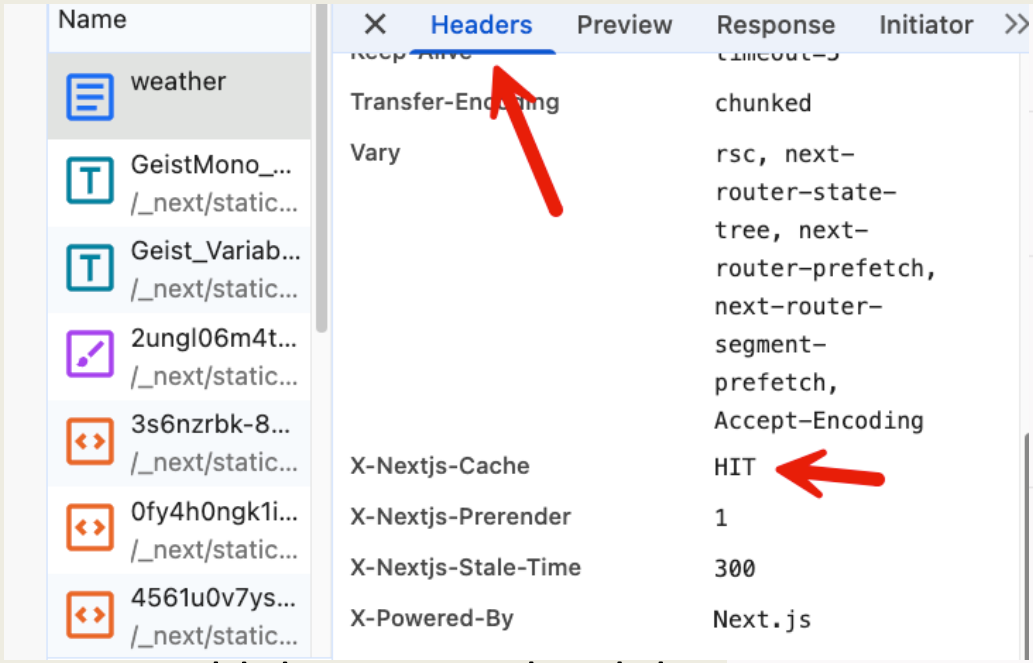
Immediately switch back to your weather tab and refresh the page.
Verification: In DevTools **Response Headers**, `x-next-cache` now displays **STALE**. The page loads instantly with the previous timestamp.

④ **Observe the Updated HIT**

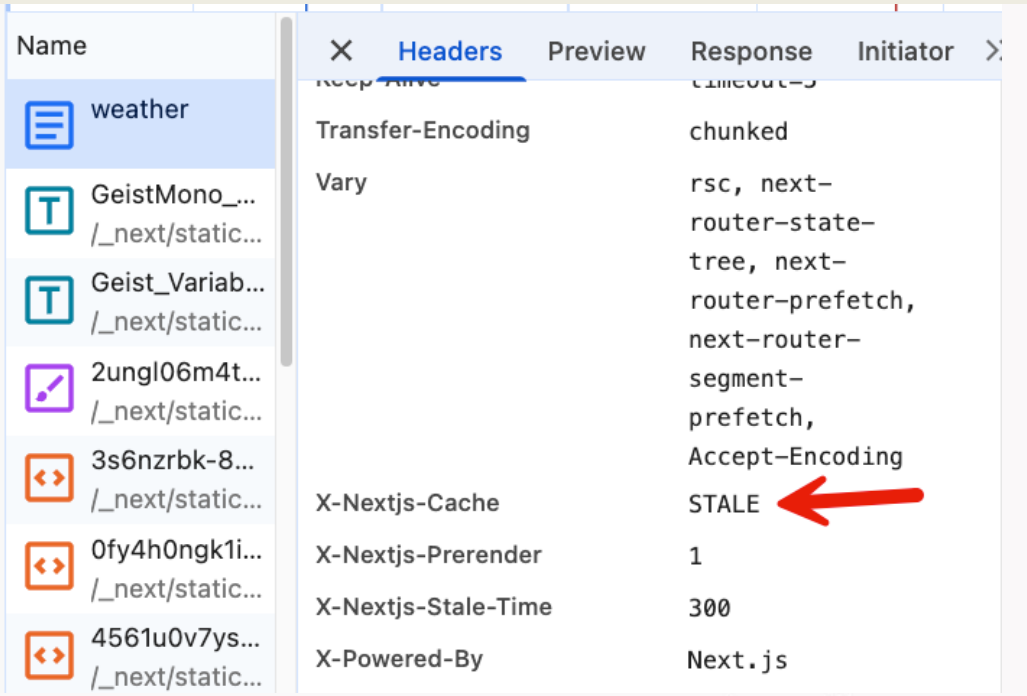
Browser Tab 1

Wait about 4 seconds (for the simulated background fetch to complete) and refresh once more.
Verification: `x-next-cache` returns to **HIT**, and the rendered timestamp updates to the new time.

x-next-cache



old data instantly while silently fetching fresh data in the background



background refresh to populate new data.

Check Server Cache



revalidatePath vs revalidateTag

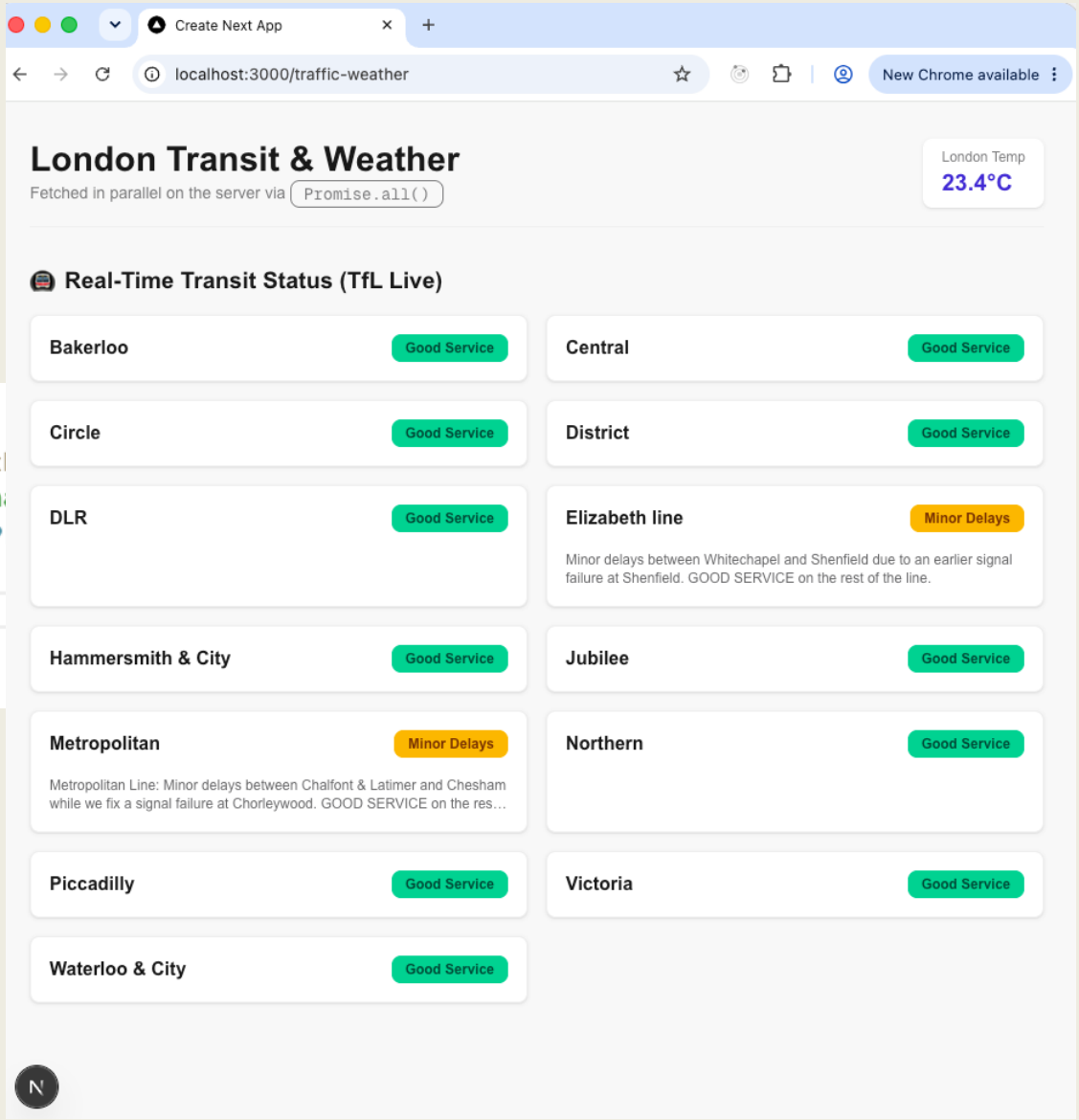
Feature	revalidatePath	revalidateTag
Scope	Targets a specific route URL (e.g., /weather)	Targets tagged fetch requests across any route
Granularity	Purges the entire page cache for that route	Purges only specific tagged data dependencies
Best For	Page-level updates, forms, post creations	Shared data (e.g., header cart count used on 10 pages)

Combine Data Fetching example

```
57
58  // --- SERVER COMPONENT ---
59  export default async function TrafficAndWeatherPage(): Promise<JSX.Element> {
60    // PARALLEL FETCHING: Triggers both external network calls simultaneously
61    const [transitData, weatherData] = await Promise.all([
62      getTfLTransitStatus(),
63      getLondonWeather(),
64    ]);
65
```

Combine Data Fetching example

```
57
58  // --- SERVER COMPONENT ---
59  export default async function TrafficAndWeat
60    // PARALLEL FETCHING: Triggers both extern
61    const [transitData, weatherData] = await P
62      getTfLTransitStatus(),
63      getLondonWeather(),
64    ]);
65
```



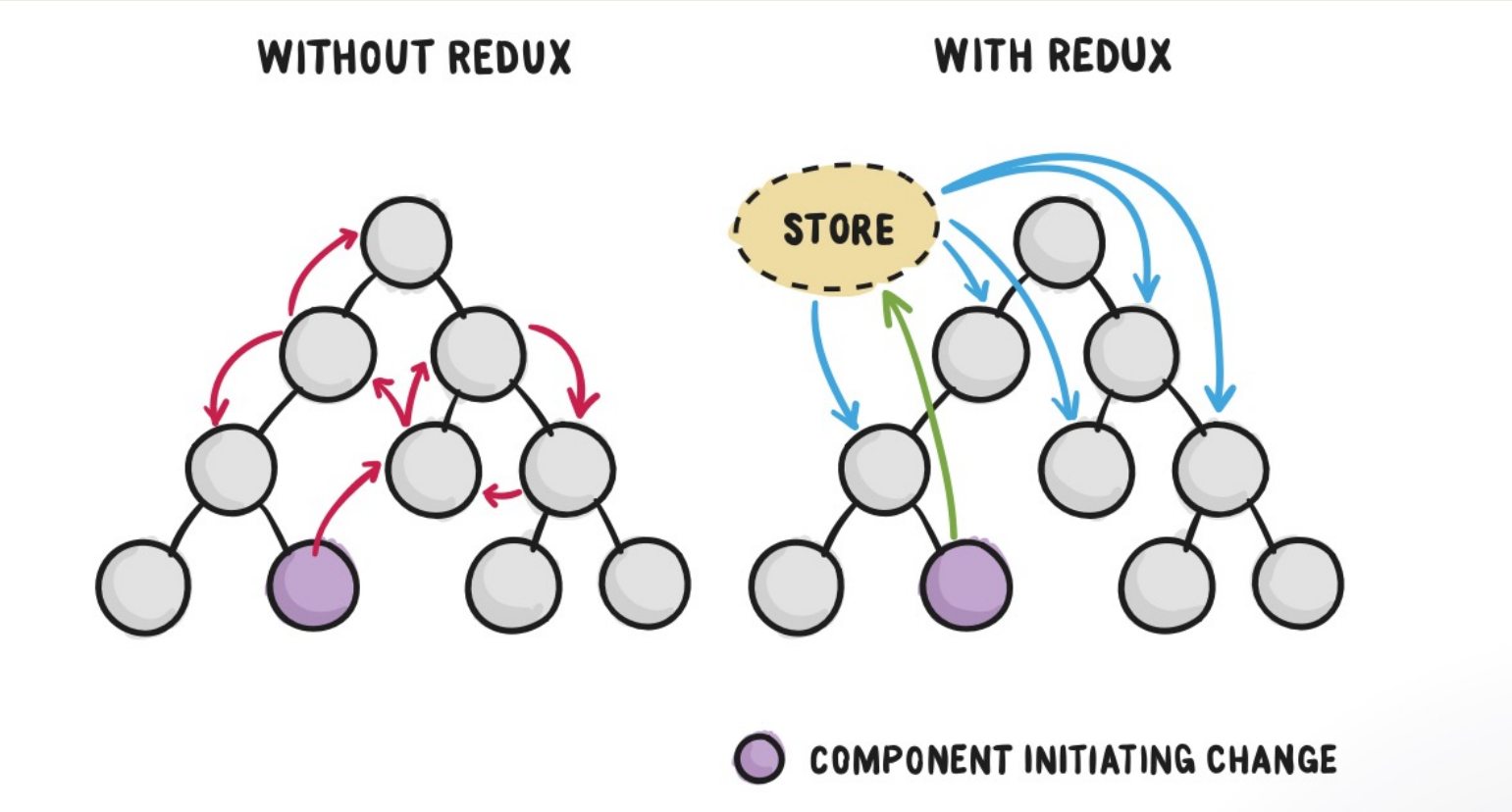
Redux Toolkit (RTK)

Standard and Efficient Global State Management for React & Next.js

Redux



- State management tool written in JavaScript
- Standalone JS library
- Supports REACT
- It allows you to manage your application's state in a single place and keep changes in your application more predictable and traceable, making it easier to understand the changes happening in your application.
- But all of these benefits come with a set of challenges.
- Developers often argue that it is overly complicated what are otherwise simple tasks.

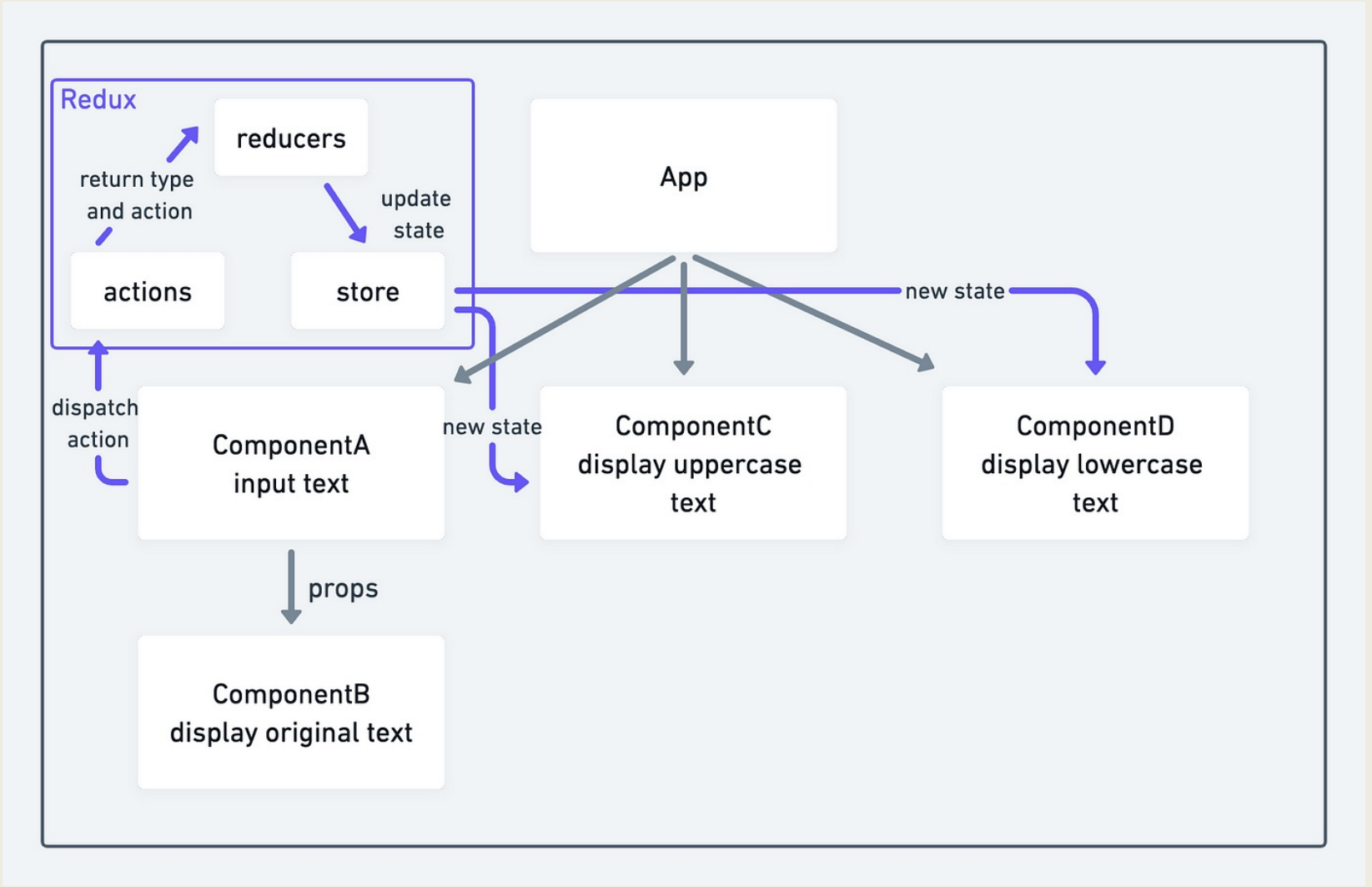


Source: https://jscrambler.com/images/redux-article-3-03.svg

Redux at a glance



- Store – a container that holds your application global state
- Action – tell the store what happens in the application
- Dispatch – dispatch the action to the store
- Reducer – store runs on reducer function and calculate the new state based on the old state and the action
- Subscribers – store notifies subscribers about the state changes to update the UI


Redux
reducers
return type
and action
update
state
actions
store
App
new state
dispatch
action
ComponentA
input text
new state
ComponentC
display uppercase
text
ComponentD
display lowercase
text
props
ComponentB
display original text

Redux Principles



1. The global state of your application is stored in an object tree within a single store.
2. State is READ-ONLY
   - The only way to change the state is to emit an action, an object, describing what happened, expressing an intent to transform the state.
3. Changes are made with pure functions
   - To specify how the state tree is transformed by actions, you write pure reducers.
   - Reducers are just pure functions that take the previous state and an action, and return the next state.
   - make sure to return new state objects, instead of mutating the previous state.

Reducers

- Remember - a reducer is a pure function that takes an action and the previous state of the application and returns the new state.
  - The action describes what happened and it is the reducer's job to return the new state based on that action
- You can start with a single reducer, and as your app grows, split it off into smaller reducers that manage specific parts of the state tree.
- Because reducers are just functions, you can control the order in which they are called, pass additional data, or even make reusable reducers for common tasks such as pagination.

Redux Toolkit (RTK)

- RTK - ‘Simplify’ version of Redux
- It is used for writing Redux application logic
- We will be using RTK for our app.

RTK Step-by-Step workflow

1. Install Packages

Install @reduxjs/toolkit and react-redux into your Next.js project.

2. Define Slices

Create slices via createSlice() with initial state, reducers & action creators.

3. Create Store

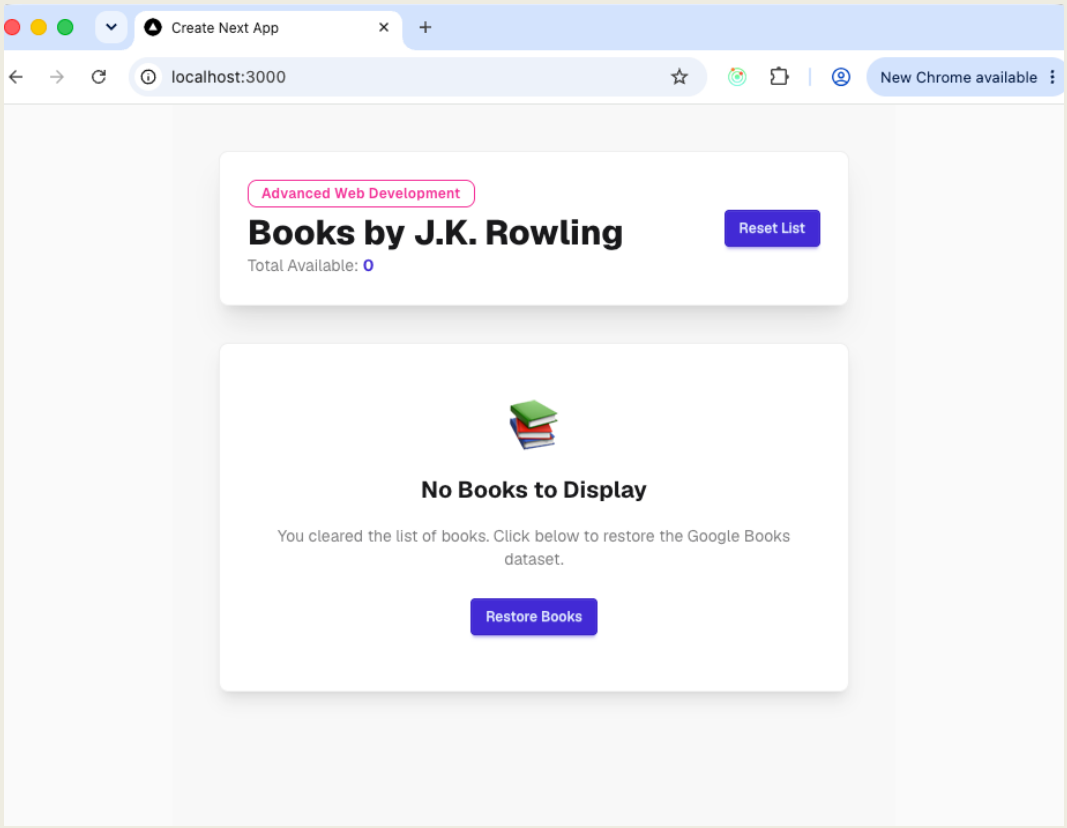
Configure central store using configureStore() with your root reducers.

4. Provide & Consume

Wrap client components with Provider and hook in using useSelector & useDispatch.

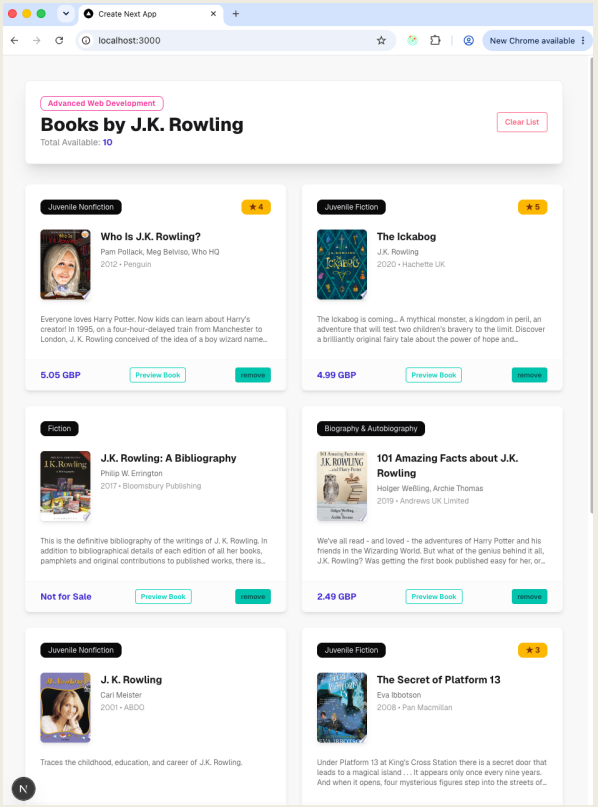
Let’s look at this example

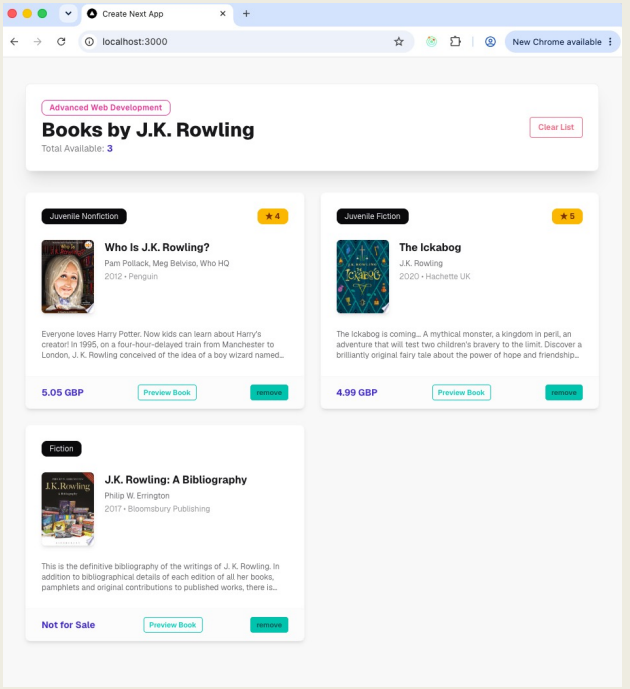
- A page rendered with a list of books, initially the book list is empty
- User can click a button to restore the book list
- User can reset the entire list
- User can delete individual book from the list



Let’s look at this example

- User can reset the entire list
- User can delete individual book from the list





Teaching code – Part 2

https://stugit.cmp.uea.ac.uk/jc/Adv-WebDev-26-27-W3-part2.git

Here we want to implement the component states using RTK

What do we need?

- Component states
  - An array to store the books
- Actions:
  - Fetch data
  - Remove book
  - Clear list

Here we want to implement the component states using RTK

What do we also need?

- Store - a container that holds the application global state
- Reducer – in Redux's world it is called 'slice'.
- Slice reducer - represents a portion of the Redux state
  - is responsible for handling actions and updating the data for a given 'slice'.
  - By convention, the naming for a given 'slice' is 'xyzSlice' where xyz refers to the item where the component states associated with, e.g. user, basket etc. You can think of it as a feature of the website.
- Dispatch – for dispatching the action

Step 1: create the ‘Slice’

- bookSlice – Reducer
  - Initial state values
  - with actions and update the state values

bookSlice – RTK createSlice()

- Function accepts
  - Slice name,
  - Initial state
  - Reducer functions - an object
  - Extra reducer functions – using builder callback
- Returns 2 things : actions and reducer

https://redux-toolkit.js.org/api/createSlice

lib > features > TS bookSlice.ts > ...

```ts
import { createSlice } from "@reduxjs/toolkit";
import { data } from "../../app/data";
import type { VolumeItem } from "../../app/data";

// 1. Define the State interface
export interface State {
  books: VolumeItem[];
}

const initialState: State = {
  books: [],
} as any;

const bookSlice = createSlice({
  name: "books",
  initialState,
  reducers: {
    clearList: (state: State) => {
      state.books = [];
    },
    resetList: (state) => {
      state.books = data.items;
    },
    deleteBook: (state, action) => {
      const id = action.payload;
      state.books = state.books.filter((book: VolumeItem) => book.id !== id);
    },
  },
});

export const { clearList, resetList, deleteBook } = bookSlice.actions;

export default bookSlice.reducer;
```

bookSlice – RTK createSlice()

```
14  const bookSlice = createSlice({
15    name: "books",
16    initialState,
17    reducers: {
18      clearList: (state: State) => {
19        state.books = [];
20      },
21      resetList: (state) => {
22        state.books = data.items;
23      },
24      deleteBook: (state, action) => {
25        const id = action.payload;
26        state.books = state.books.filter((book: VolumeItem) => book.id !== id);
27      }
28    },
29    extraReducers: (buider) => {}
30  });
31
```

- Function accepts
  - Slice name
  - Initial state
  - Reducer functions - an object
- Extra reducer functions – using builder callback
  - Use for external resources e.g. fetching data
  - First - create the fetch function using ‘createAsyncThunk’

Example: fetchUser - RTK createAsyncThunk()

- Function accepts
  - Action name as String
  - Callback function which returns a promise

```
11  export const fetchUsers = createAsyncThunk(
12    "users/fetchUsers",
13    async (name, thunkAPI) => {
14      try {
15        const response = await axios.get(URL);
16        return response.data.results;
17      } catch (error) {
18        console.log(error);
19        return thunkAPI.rejectWithValue(error.response);
20      }
21    }
22  );
```

https://redux-toolkit.js.org/api/createAsyncThunk

Example: userSlice – RTK createSlice()

- Function accepts
  - Extra reducer functions – using builder callback
    - Use for external resources e.g. fetching data
    - Maps with Promise – pending, fulfilled and rejected
      - builder.addCase()
      - A function to update the new state value

```
34    extraReducers: (builder) => {
35      builder.addCase(fetchUsers.pending, (state) => {
36        state.isLoading = true;
37      });
38      builder.addCase(fetchUsers.fulfilled, (state, action) => {
39        console.log(state);
40        console.log(action.payload);
41        state.isLoading = false;
42        state.randomUsers = action.payload;
43        console.log(state.randomUsers.length);
44      });
45      builder.addCase(fetchUsers.rejected, (state) => {
46        state.isLoading = false;
47      });
48    },
49  });
```

bookSlice – RTK createSlice()

- Function accepts
  - Slice name –
  - Initial state
  - Reducer functions - an object
  - Extra reducer functions – using builder callback
- Export actions
- Export reducer

```
32  export const { clearList, resetList, deleteBook } = bookSlice.actions;
33
34  export default bookSlice.reducer;
35
```

Step 2: create a ‘store’ to hold the application global states

- store.ts
- Use configureStore to initialise the reducer
- Export the recommended types

https://redux-toolkit.js.org/usage/nextjs

lib > TS store.ts > ...

```ts
import { configureStore } from "@reduxjs/toolkit";
import "@/lib/features/bookSlice";
import bookReducer from "./features/bookSlice";

export const makeStore = () => {
  return configureStore({
    reducer: { books: bookReducer },
  });
};

// Infer the type of makeStore
export type AppStore = ReturnType<typeof makeStore>;
// Infer the `RootState` and `AppDispatch` types from the store itself
export type RootState = ReturnType<AppStore["getState"]>;
export type AppDispatch = AppStore["dispatch"];

```

Step 3: Client-side ’Provider’

- Code a client-side Provider
  - Import store

app > BookProvider.tsx > ...

```tsx
"use client";
import { useRef } from "react";
import { Provider } from "react-redux";
import { makeStore, AppStore } from "../lib/store";

import React from "react";

const BookProvider = ({ children }: { children: React.ReactNode }) => {
  const storeRef = useRef<AppStore>(undefined);
  if (!storeRef.current) {
    // Create the store instance the first time this renders
    storeRef.current = makeStore();
  }

  return <Provider store={storeRef.current}>{children}</Provider>;
};

export default BookProvider;

```

useRef hook

- In React, useRef hook allows you to store a value or reference that persists across component re-renders without causing a re-render when that value changes.

- Think of it as a "secret pocket" attached to your component instance. Anything you put in that pocket stays there for as long as the component lives, and changing what's inside won't trigger React to redraw the screen.

- React returns a simple JavaScript object with a single property: .current.

Step 4: Consume Provider

- Add the Book Provider to layout.tsx

```
4  import BookProvider from "./BookProvider";
```

```
27      <body className="min-h-full flex flex-col">
28        <BookProvider>{children}</BookProvider>
29      </body>
```

Step 5: use ‘dispatch’ to dispatch the action

app > Books.tsx > Books

```
1  "use client";
2
3  import { clearList, resetList, deleteBook } from "@/lib/features/bookSlice";
4  import { useAppDispatch, useAppSelector } from "@/lib/hooks";
5
6  const Books = () => {
7    const { books } = useAppSelector((store) => store.books);
8    const dispatch = useAppDispatch();
9
10   const handleClear = () => {
11     dispatch(clearList());
12   };
13
14   const reset = () => {
15     dispatch(resetList());
16   };
17
18   const remove = (id: any) => {
19     dispatch(deleteBook(id));
20   };
21
```

Recap

- Reducer – bookSlice
  - createSlice and createAsyncThunk from RTK
  - Global state
    - Initial state
  - Reducers
    - Action and update the new state value
  - Extra Reducers
    - Buidler – addcase
    - Promise – pending, fulfilled and rejected
    - Action to update the new state value
- Store - a container that holds the application global state
  - createStore() from RTK
- Dispatch – component
  - for dispatching the action

Practical Exercise and homework

- Lab 3.0 – useEffect hook and Axios API
- Lab 3.1 – Redux Toolkit (RTK)

Next Session

- Auth0 & Google Provider